RFID 开发文档

[V2.19]

目录

1. 常用方法和流程 6
1.1 清单 6
1.2 说明 6
1.3 流程 6
2. RFID API 接口 9
2.1 标初始化和连接 9
2.1.1 一体机和 UART 设备 9
2.1.2 蓝牙设备初始化(非蓝牙设备请忽略本章) 10
2.1.3 isConnected【判断是否连接上 rfid 模块】 11
2.2 RFID 标签盘点和回调 11
2.2.1 startInventoryWithTimeout【启动盘点】 12
2.2.2 startInventory【启动盘点】 13
2.2.3 registerCallback【注册盘点结果的回调】 15
2.2.4 stopInventory【停止盘点】 16
2.2.5 scanRfid【询查一次标签】 16
2.2.6 inventorySingle【单个标签查询】 16
2.3 盘点场景模式选择 17
2.3.1 setInventorySceneMode【设置盘点场景模式】 17
2.3.2 getInventorySceneMode【获取盘点场景模式】 18
2.4 盘点存储区设置 19
2.4.1 setQueryMemoryBank【设置查询存储区模式】 19
2.4.2 getQueryMemoryBank【获取查询模式】 21
2.5 盘点参数设置 22
2.5.1 setOutputPower【设置输出功率】 22
2.5.2 getOutputPower【查询输出功率】 23
2.5.3 setProfile【设置 BLF、Miller、Tari、PIE 】 23
2.5.4 getProfile【设置 BLF、Miller、Tari、PIE 】 25
2.5.5 setInventoryWithSession【设置 Session】 25
2.5.6 getInventoryWithSession【获取 Session】 26
2.5.7 setInventoryWithTarget【设置盘点时使用的 Target】 27
2.5.8 getInventoryWithTarget【获取盘点会话目标值】 28
2.5.9 setInventoryWithStartQvalue【设置盘点时的初始 Q-value】 28
2.5.10 getInventoryWithStartQvalue【获取盘点的初始 Q-value】 29
2.5.11 setInventoryWithPassword【设置盘点时的询查密码】 29
2.5.12 getInventoryWithPasssword【获取盘点时的询查密码】 30
2.5.13 setInventoryRssiLimit【RSSI 过滤】 30
2.5.14 isSupportInventoryRssiLimit【是否支持限制 RSSI】 31
2.5.15 setRssiInDbm【设置盘点返回信号的类型】 31
2.5.16 addMask【添加掩码】 32
2.5.17 clearMask【清除掩码】 34
2.6 标签操作 34

2.6.1 readTag【读取标签】 34
2.6.2 writeTag【写标签】 36
2.6.3 writeTagEpc【写标签 EPC 区】 37
2.6.4 writeEpc【随机改写一张标签的 EPC】 38
2.6.5 lockTag【锁定标签】 38
2.6.6 killTag【销毁标签】 40
2.6.7 readDataByTid【通过 tid 读取数据】 41
2.6.8 writeTagByTid【通过 tid 向各存储区写数据】 41
2.6.9 lockByTID【通过 TID 锁定标签】 42
2.6.10 killTagByTid【通过 TID 销毁标签】 44
2.6.11 writeTagEpcByTid【通过 TID 写 EPC】 44
2.6.12 maskReadTag【带掩码读取标签】 45
2.6.13 maskWriteTag【带掩码写标签】 48
2.6.14 readTagExt【读取大容量标签】 52
2.6.15 writeTagExt【写大容量标签】 53
2.6.16 eraseTag【擦除标签】 55
2.6.17 findEpc【查找标签】 56
2.6.18 lightUpLedTag【点亮指定的 LED 标签】 56
2.6.19 startInventoryLed【询查 ELD 标签】 57
2.6.20 stopInventoryLed【停止询查 ELD 标签】 57
2.7 频段和其他设置 58
2.7.1 setWorkRegion【设置区域频段】 58
2.7.2 getWorkRegion【获取工作频率】 59
2.7.3 setFrequencyRegion【设置频率区域】 59
2.7.4 getFrequencyRegion【获取频率区域】 60
2.7.5 getFirmwareVersion【获取固件版本】 61
2.7.6 setTagFocus【设置 TagFocus 命令】 61
2.7.7 getTagFocus【读取 TagFocus 命令】 62
2.7.8 getModuleFirmware【获取模块固件信息】 62
2.7.9 GetReaderType【获取模块类型】 62
2.7.10 getReaderTemperature【查询内部温度】 63
2.7.11 setCustomRegion【设置自定义频段】 63
2.7.12 getCustomRegion【获取自定义频段】 64
2.7.13 getEx10Version [获取 E 系列芯片信息] 64
2.7.14 enableLog【开启或者关闭日志】 65
2.8 蜂鸣器设置 66
2.8.1 setBeepEnable【设置开启蜂鸣器】 66
2.8.2 setBeepVolume【设置蜂鸣器音量】 66
2.8.3 startBeep【控制蜂鸣器响一次】 66
3 固件升级 68
3.1 一体机设备升级(包含 RFG91 UART) 68
3.1.1 updateReaderFirmwareByFile【RFID 模块升级--通过读取文件】 68
3.1.2 updateReaderFirmwareByByte【RFID 模块升级--通过字节流】 70

3.1.3 updateEx10ChipFirmwareByFile【RFID 芯片升级--通过读取文件】..... 71
3.1.4 updateEx10ChipFirmwareByByte【RFID 芯片升级--通过字节流】..... 72
3.2 蓝牙设备升级..... 74
3.2.1 updateBLEReaderFirmwareByFile【RFID 模块升级--通过读取文件】.... 74
3.2.2 updateBLEReaderFirmwareByByte【RFID 模块升级--通过读取文件】.... 76
3.2.3 updateBLEEx10ChipFirmwareByFile【RFID 模块升级--通过字节流】.... 77
3.2.4 updateBLEEx10ChipFirmwareByByte【RFID 模块升级--通过字节流】.... 79
4.蓝牙、UART 分体设备接口说明(非一体设备)..... 81
4.1 扫描条码的结果获取..... 81
4.2 设备状态的回调监听..... 82
4.2.1 setKeyEventListener【注册按键事件监听】..... 82
4.2.2 setBatteryGripListener【注册设备充电状态、电池电量变化的监听】. 83
4.3 setSwitchCallback【监听 RFID 模块之间的切换】..... 84
4.3 startScanBarcode【开始扫描条码】..... 86
4.4 getElectricQuantity【获取电池电量】..... 86
4.5 getIsChange【获取电池充电状态】..... 86
4.6 getVersionSystem【获取设备版本号】..... 87
4.7 getVersionBLE【获取蓝牙版本号】..... 87
4.8 getDeviceSN【获取设备 SN 号】..... 88
4.9 setBeepRange【设置蜂鸣器音量】..... 88
4.10 getBeepRange【获取蜂鸣器音量】..... 88
4.11 setSleepTime【设置休眠时间】..... 89
4.12 getSleepTime【获取休眠时间】..... 89
4.13 setPowerOffTime【设置超时关机时间】..... 89
4.14 getPowerOffTime【获取蜂鸣器音量】..... 90
4.15 getBLEMac【获取蓝牙设备的 MAC 地址】..... 90
4.16 setOfflineModeOpen【设置离线模式】..... 91
4.17 getOfflineModeOpen【获取离线模式开启状态】..... 91
4.18 setOfflineTransferClearData【设置离线模式传输后是否自动清除数据】..... 92
4.19 getOfflineTransferClearData【获取离线模式下自动清除数据的状态】..... 92
4.20 setOfflineTransferDelay【设置离线模式下传输数据，数据之间的发送间隔时间】..... 92
4.21 getOfflineTransferClearData【获取离线模式下自动清除数据的状态】..... 93
4.22 getOfflineQueryNum【获取离线数据的数量】..... 93
4.23 getOfflineQueryMem【获取离线数据占用的内存百分比】..... 94
4.24 offlineManaulClearData【主动清除离线模式下保存的数据】..... 94
4.25 offlineStartTransferRFID【主动清除离线模式下保存的 RFID 数据】..... 95
4.26 offlineStartTransferScan【主动清除离线模式下保存的 RFID 数据】..... 95
4.27 modeResetFactory【设备固件恢复出厂设置】..... 96
附录 1：常见错误码..... 97
附录 2：频段说明..... 99
附录 3: RSSI 参数对照表..... 101
附录 4: 区域对照表..... 103

附录 5: APP 打包的混淆规则 105
备注 106
额外 API 说明 106

1. 常用方法和流程

1.1 清单

- **硬件要求:**确保是本司提供的设备及 RFID 模块。
- **SDK 内容:** 包含一个 aar 包文件， URFIDLibrary-xxxxxx.aar 与硬件通信，提供一套便于开发的接口给应用层调用;
- **Demo:** UHFDemo 是使用超高频的一个 demo 示例，可参考开发使用.
- **文档:** 《RFID 开发文档 v2.19.docx》 是 SDK 开发指南及接口方法使用说明.

1.2 说明

Android 平台下的 RFID SDK ，开发包 URFIDLibrary-xxxxxx.aar，开发者可以通过 SDK 内的 API 方法，直接调用 RFID 模块，效率高效、简单。在初始化成功之后，可正常使用其接口方法。

1.3 流程

集成的步骤:

1.添加依赖

导入 URFIDLibrary-xxxxxx.aar 到你的 Android 工程:

```
implementation files('libs/URFIDLibrary-xxxxxx.aar')
```

2.初始化 **SDK**

注意：SDK 的方法一般都会阻塞主线程，所以建议在异步线程中调用 API，然后发消息到主线程更新 UI。同时，API 需要依次调用，上一个 API 执行完后，再执行下一个指令。

- 第一次进入 APP，调用初始化方法： init(Context context,InitListener listener)进入待机状态（低耗电）:
  RFIDSDKManager.getInstance().int init(Context context,InitListener listener);

- 退出应用时调用断开 RFID 模块的方法，进入完全不工作状态，此状态下 RFID 模块不耗电,下次要再次使用需要再次调用 init(Context context,InitListener listener):

  RFIDSDKManager.getInstance().release();

4.盘点操作

- 开始盘点:
  RFIDSDKManager.getInstance().getRfidManager().startInventoryWithTimeout(int timeout)
  或
  RFIDSDKManager.getInstance().getRfidManager().startRead();
  （盘点状态耗电较大，如果工作完成，请及时调用停止盘点的方法）
- 停止盘点:
  RFIDSDKManager.getInstance().getRfidManager().stopInventory();
- 盘点结果的数据回调:
  RFIDSDKManager.getInstance().getRfidManager().registerCallback(new IRfidCallback() {
  @Override
  public void onInventoryTag(String EPC, String Data, int rssi) {}
  @Override
  public void onInventoryTagEnd() {}});
- 回调方法和参数说明:

1> onInventoryTag【结果回调方法】

定义	void onInventoryTag(String EPC, String Data, int RSSI);		
说明	该方法会在开启盘点后，通过回调该方法返回盘点到的标签信息。		
参数	名称	类型	备注
	EPC	String	盘点到的标签的 epc（不包括 CRC、PC）
	Data	String	混合查询模式下返回的对应区域的数据，比如：TID 数据或 USER 区数据； 混合查询可通过 setQueryMemoryBank(QueryMemBank mode) 方法设置。
	RSSI	int	标签的信号强度，值的范围为大于 0 的数值，如果要转换成 dBm 值，可参考 【附录 3：RSSI 参数对照表】或者使用工具类转换： int dBm = RssiToDBmUtils.dBmConver(int RSSI);

2> onInventoryTagEnd【盘点结束回调】

定义	void onInventoryTagEnd();
说明	此方法盘点结束的时候调用。

2. RFID API 接口

2.1 标初始化和连接

2.1.1 一体机和 UART 设备

2.1.1.1 init【模块初始化】

定义	void init(Context context,InitListener listener);		
说明	RFID 初始化.		
参数	名称	类型	备注
	context	Context	应用上下文

	listener	InitListener	监听初始化成功或失败的回调
			`new InitListener() {`
			`@Override`
			`public void onStatus(boolean status) {`
			`}`
			`}`
			status :
			true:初始化成功
			false:初始化失败

返回			
参考代码	无。		

2.1.1.2 release【断开模块】

定义	void release();		
说明	断开 RFID 模块， RFID 模块进入完全不工作状态，应用退出时需要调用。		
参数	名称	类型	备注
返回			

参考代码	无。

2.1.2 蓝牙设备初始化(非蓝牙设备请忽略本章)

2.1.2.1 initBTToMac【蓝牙设备初始化】

定义	void initBTtoMac(Context context, String mac, InitListener listener);		
说明	连接带 RFID 功能的蓝牙设备。		
参数	名称	类型	备注
	context	Context	上下文
	mac	String	需要连接的蓝牙设备的 MAC 地址，例如：41:BB:00:F8:DB:51
	listener	InitListener	监听初始化成功或失败的回调 `new InitListener() { @Override public void onStatus(boolean status) { } }` status : true:初始化(连接)成功 false:初始化(连接)失败
返回	无		
参考代码	无。		

2.1.2.2 release【释放资源】

定义	void release();		
说明	释放资源，断开蓝牙设备。		
参数	名称	类型	备注
返回	无		
参考代码	无		

2.1.3 isConnected【判断是否连接上 rfid 模块】

定义	boolean isConnected();		
说明	判断是否以及连上了 RFID 模块。		
参数	名称	类型	备注
返回 (boolean)	true：已连接 ； false：未连接 。		
参考代码	无		

2.2 RFID 标签盘点和回调

在设备初始化之后，调用以下 RfidManager 来控制后续的 API ：

```
RFIDSDKManager.getInstance().getRfidManager()
```

2.2.1 startInventoryWithTimeout【启动盘点】

<table>
<tr><td>定义</td><td colspan="3">int startInventoryWithTimeout(int timeout);</td></tr>
<tr><td>说明</td><td colspan="3">启动盘点，这里的传参 timeout 单位为 seconds。在盘点执行到 timeout 时间，会自动停止盘点，同时会在回调方法的 onInventoryTagEnd() 回调通知，如果未到 timeout 时间，想要停止，也可以调用 2.2.4 stopInventory() 进行手动停止。
结果在 IRfidCallback()回调返回。</td></tr>
<tr><td>参数</td><td>名称</td><td>类型</td><td>备注</td></tr>
<tr><td></td><td>timeout</td><td>int</td><td>超时时间，单位：秒，必须大于或等于 0.并且不能超过 36000 秒：

设置 0 ：
则表示一直盘点，必须要调用 stopInventory()才会停止；

设置 0< timeout <= 36000：
则会根据 timeout 时间，到时间自动停止，期间也可以调用 stopInventory()进行提前停止。</td></tr>
<tr><td>返回
(int)</td><td colspan="3">成功：RfidErrorConstants.SUCCESS;
失败：其他(错误码信息详见:<附录 1：常见错误码>)。</td></tr>
<tr><td>参考代码</td><td colspan="3">RFIDSDKManager.getInstance().getRfidManager().startInventoryWithTimeout(300);

如果要监听 RFID 手柄的按键来触发盘点方法，可以使用如下方式：
1.一体机设备，监听 android 标准按键事件：

@Override
public boolean dispatchKeyEvent(KeyEvent event) {
if ((event.getKeyCode() == 523 || event.getKeyCode() == 515)
&& event.getRepeatCount() == 0) {
//TODO The key value of the PTT button on the left side of DT610 is 515
if (event.getAction() == KeyEvent.ACTION_DOWN) {
//start Inventory

//RFIDSDKManager.getInstance().getRfidManager().startInventoryWithTimeout(300);
return true;
} else if (event.getAction() == KeyEvent.ACTION_UP) {
//stop Inventory</td></tr>
</table>

```
                //RFIDSDKManager.getInstance().getRfidManager().stopInventory();
                return true;
            }
        }
        return super.dispatchKeyEvent(event);
    }
```

2.蓝牙设备，监听方法的回调

```
GripDeviceManager.getInstance().setKeyEventListener(new KeyEventListener() {
                @Override
                public void event(int keyCode, boolean isDown) {
                    //keycode == BTKeyEvent.BT_SCAN   The handle button was pressed
                    //isDown  true: down   false: up
                    if (keyCode ==  BTKeyEvent.BT_SCAN) {
                        if (isDown  ) {
                         //start Inventory
                        //RFIDSDKManager.getInstance().getRfidManager().startRead();
                        } else {
                            //stop Inventory

//RFIDSDKManager.getInstance().getRfidManager().stopInventory();
                        }
                    }
                }
            });
```

2.2.2 startInventory【启动盘点】

定义	int startInventory();		
说明	启动盘点； 启动后，必须要通过 stopInventory()来停止。 盘点到的 Tag 结果在 IRfidCallback()回调方法返回。		
参数	名称	类型	备注
返回 (int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	RFIDSDKManager.getInstance().getRfidManager().startInventory();		

如果要监听 RFID 手柄的按键来触发盘点方法，可以使用如下方式：
1.一体机设备，监听 android 标准按键事件：

```
@Override
 public boolean dispatchKeyEvent(KeyEvent event) {
     if (event.getKeyCode() == 523 && event.getRepeatCount() == 0) {
         //TODO  The Grip key value of the key is 523
         if (event.getAction() == KeyEvent.ACTION_DOWN ) {
             //start Inventory
             //RFIDSDKManager.getInstance().getRfidManager().startInventory();
             return true;
         } else if (event.getAction() == KeyEvent.ACTION_UP ) {
             //stop Inventory
             //RFIDSDKManager.getInstance().getRfidManager().stopInventory();
             return true;
         }
     }
     return super.dispatchKeyEvent(event);
 }
```

2.蓝牙设备，监听方法的回调

```
GripDeviceManager.getInstance().setKeyEventListener(new KeyEventListener() {
            @Override
            public void event(int keyCode, boolean isDown) {
                //keycode == BTKeyEvent.BT_SCAN   The handle button was pressed
                //isDown  true: down   false: up
                if (keyCode ==  BTKeyEvent.BT_SCAN) {
                    if (isDown  ) {
                     //start Inventory
                    //RFIDSDKManager.getInstance().getRfidManager().startInventory();
                    } else {
                        //stop Inventory

//RFIDSDKManager.getInstance().getRfidManager().stopInventory();
                    }
                }
            }
        });
```

2.2.3 registerCallback【注册盘点结果的回调】

定义	void registerCallback(IRfidCallback cb);		
说明	注册回调（标签数据将在注册回调中回调）。		
参数	名称	类型	备注
	cb	IRfidCallback	回调
返回 (void)	无		
参考代码	无		

2.2.3.1 onInventoryTag()

定义	void onInventoryTag(String EPC, String Data, int RSSI);		
说明	在开启盘点后，通过该方法回调盘点到的标签信息。		
参数	名称	类型	备注
	EPC	String	盘点到的标签的 EPC（不包含 CRC、PC）
	Data	String	混合查询模式下返回的对应区域的数据，比如：TID 数据、USER 区数据或保留区数据； 混合查询可通过 setQueryMemoryBank(QueryMemBank mode) 方法设置。
	RSSI	int	标签的信号强度，值的范围为大于 0 的数值，如果要转换成 dBm 值，可参考 【附录 3：RSSI 参数对照表】

2.2.3.2 onInventoryTagEnd

定义	void onInventoryTagEnd();
说明	此方法盘点结束的时候调用。

2.2.4 stopInventory【停止盘点】

定义	int stopInventory();		
说明	停止盘点。		
参数	名称	类型	备注
返回(int)	成功：RfidErrorConstants.SUCCESS; 失败：(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

2.2.5 scanRfid【询查一次标签】

定义	void scanRfid();		
说明	读取一次后自动停止，可能读到 0 个或多个标签，结果在IRfidCallback()回调返回。		
参数	名称	类型	备注
返回(void)			
参考代码	无		

2.2.6 inventorySingle【单个标签查询】

定义	int inventorySingle();

说明	每次调用最多只读取一个标签，如果附近没有标签，也有可能读不到。 结果在 IRfidCallback()回调返回。		
参数	名称	类型	备注
返回	0：调用成功； 其他：调用失败(错误码信息详见:<附录 1：常见错误码>)。 读取结果在 onInventoryTag() 回调返回对应结果。		
参考代码	无。		

2.3 盘点场景模式选择

2.3.1 setInventorySceneMode【设置盘点场景模式】

定义	int setInventorySceneMode(InventorySceneMode mode);		
说明	几种不同场景下的模式，可以根据自己的场景，选择对应的模式；如果需要自定义设置，也可以通过本文档中的《2.5 盘点参数设置》进行自定义设置。 注意：如果在非自定义模式(InventorySceneMode.CUSTOM_MODE)下，参数 Session、Qvalue、Target、Profile 都不可设置。 每次设置模式后，功率会变更成对应模式下的默认值，如果之前手动设置过功率的大小，在设置模式后，需要再次设置一次自己期望的的功率值。		
参数	名称	类型	备注
	mode	InventorySceneMode	盘点场景模式，请参考下面的：**盘点场景模式说明。**

返回 (int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	RFIDSDKManager.getInstance().getRfidManager().setInventorySceneMode(InventorySceneMode.BATTERY_LIFE);		

盘点场景模式说明：

盘点场景模式 (InventorySceneMode)	适用场景	核心功能
BATTERY_LIFE	低功耗场景	延长设备续航时间
FASTEST_READ	快速读取全标签	高速盘点，适用于 E 系列芯片
REPEAT_READ	测距、重复标签统计	频繁(重复)读取并累计标签数量
BALANCED_PERFORMANCE	均衡模式	平衡性能和续航
CUSTOM_MODE	自定义参数设置	参数可自行配置配置

2.3.2 getInventorySceneMode【获取盘点场景模式】

定义	InventorySceneMode getInventorySceneMode();		
说明	获取之前设置过的盘点场景模式。		
参数	名称	类型	备注
返回 (InventorySceneMode)	InventorySceneMode ： InventorySceneMode.BATTERY_LIFE InventorySceneMode.FASTEST_READ InventorySceneMode.REPEAT_READ InventorySceneMode.BALANCED_PERFORMANCE InventorySceneMode.CUSTOM_MODE		
参考代码	InventorySceneMode mode = RFIDSDKManager.getInstance().getRfidManager().getInventorySceneMode();		

2.4 盘点存储区设置

2.4.1 setQueryMemoryBank【设置查询存储区模式】

定义	int setQueryMemoryBank(QueryMemBank mode,int startAddress,int readLen);		
说明	设置盘点时查询标签的存储区。		
参数	名称	类型	备注
	mode	QueryMemBank	请看下方的 **QueryMemBank 说明。**
	startAddress	int	要盘点的内存区域的起始位置，以字（Word）地址（Word Address） 为单位((1 字长 = 16 bits) 。
	readLen	int	盘点时要读取的字(Word)长度(1 字长 = 16 bits) 。
返回(int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	RFIDSDKManager.getInstance().getRfidManager().setQueryMemoryBank(QueryMemBank.EPC,0,0); RFIDSDKManager.getInstance().getRfidManager().setQueryMemoryBank(QueryMemBank.EPC_USER,startAddress,readLen);		

QueryMemBank 说明：

QueryMemBank	读取的 Bank	说明

EPC	EPC 查询	参数设置： startAddress = 0, readLen = 0 读取行为： 自动读取默认长度的 EPC 内容（不包含 CRC 和 PC 头）
EPC_TID	EPC+TID 查询	参数设置： startAddress = TID 起始字地址（0-5） readLen = TID 读取长度（1-6 字） 数据返回： onInventoryTag(String EPC, String Data, int RSSI) EPC → onInventoryTag 的 EPC 参数 TID 数据 → Data 参数（从 startAddress 开始，长度 readLen 字）
EPC_USE	EPC+USER 查询	参数设置： startAddress = USER 起始字地址（大于或等于 0） readLen = USER 读取长度（1-32 字） 数据返回： onInventoryTag(String EPC, String Data, int RSSI) EPC → EPC 参数 USER 数据 → Data 参数
EPC_RESERVED	EPC+RESERVED（Password）查询	参数设置： startAddress = RESERVED 起始地址（0-3） readLen = 密码读取长度（1-4 字） 数据返回： onInventoryTag(String EPC, String Data, int RSSI) EPC → EPC 参数 RESERVED 数据 → Data 参数
FASTTID	FASTTID 查询	Impinj FastTID 快速模式 参数设置：

		startAddress = 0, readLen = 0（参数固定） 数据返回： onInventoryTag(String EPC, String Data, String int RSSI) EPC → EPC 参数 自动读取完整 96 位 TID（字地址 0-5） TID 数据 → Data 参数 兼容性： 仅支持 Impinj Monza®系列标签
EPC_EPC	EPC+EPC 查询	EPC + EPC 双区查询 参数设置： startAddress = 第二 EPC 起始地址 readLen = 第二 EPC 读取长度 数据返回： onInventoryTag(String EPC, String Data, int RSSI) 第一 EPC（默认长度） → EPC 参数 第二 EPC（自定义段） → Data 参数 典型应用： 读取协议控制位（PC）或部分 EPC 验证

2.4.2 getQueryMemoryBank【获取查询模式】

定义	QueryMemBank getQueryMemoryBank();

说明	获取盘点数据区域的模式		
参数	名称	类型	备注

返回
(int)
	返回的对应模式值：
	QueryMemBank queryMemBank ：
	
	queryMemBank .getStartAddress();//要读取/写入的内存区域的起始位置，以字（Word）地址（Word Address） 为单位。
	queryMemBank .getReadLength();//读取的字长度(一个字长为 16 bits)		

参考代码	QueryMemBank queryMemBank =
	RFIDSDKManager.getInstance().getRfidManager().getQueryMemoryBank();
	int startAdd = queryMemBank .getStartAddress();
	int readLen = queryMemBank .getReadLength();		

2.5 盘点参数设置

2.5.1 setOutputPower【设置输出功率】

定义	int setOutputPower(int outputPower);		
说明	设置功率		
参数	名称	类型	备注
	outputPower	int	设备的输出功率：

			模块支持的最大功率值可通过下面方法获取： int maxPower = RFIDSDKManager.getInstance().getRfidManager().getSupportMaxOutputPower(); maxPower 即为支持的最大功率值。
返回 (int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

2.5.2 getOutputPower【查询输出功率】

定义	int getOutputPower();		
说明	查询输出功率。		
参数	名称	类型	备注
返回 (int)	返回输出功率(小于 0 则获取失败)		
参考代码	无		

2.5.3 setProfile【设置 BLF、Miller、Tari、PIE 】

定义	int setProfile(RfidProfile profile);

<table>
<tr><td>说明</td><td colspan="3">设置链路信息：BLF(Backscatter Link Frequency)、Miller、Tari、Pie
该方法是 BLF、Miller、Tari、PIE 参数的固定组合。

注意：
如果处于非自定义模式(InventorySceneMode.CUSTOM_MODE)，该参数不可设置；
如果要设置，请先通过 2.3.1 setInventorySceneMode()设置盘点模式为自定义模式(InventorySceneMode.CUSTOM_MODE)。</td></tr>
<tr><td rowspan="2">参数</td><td>名称</td><td>类型</td><td>备注</td></tr>
<tr><td>param</td><td>RfidProfile</td><td>芯片类型可通过 2.7.13 getEx10Version［获取 Ex10 芯片信息］获取。

不同芯片支持的 RfidProfile 组合参考下方表格。</td></tr>
<tr><td>返回（int）</td><td colspan="3">成功：RfidErrorConstants.SUCCESS；
失败：其他(错误码信息详见:<附录 1：常见错误码>)。</td></tr>
<tr><td>参考代码</td><td colspan="3">RFIDSDKManager.getInstance().getRfidManager().setProfile(RfidProfile.PROFILE_5);</td></tr>
</table>

不同芯片支持的 Profile：

RfidProfile	芯片支持	参数组合
PROFILE_3	E710, E510, E310	PR_ASK, PIE: 2.0, Tari: 20.0μs, BLF: 320kHz, Miller=2
PROFILE_5	E710, E510, E310	PR_ASK, PIE: 2.0, Tari: 20.0μs, BLF: 320kHz, Miller=4
PROFILE_7	E710, E510, E310	PR_ASK, PIE: 2.0, Tari: 20.0μs, BLF: 250kHz, Miller=4
PROFILE_12	E710, E510, E310	PR_ASK, PIE: 2.0, Tari: 15.0μs, BLF: 320kHz, Miller=2
PROFILE_13	E710, E510, E310	PR_ASK, PIE: 2.0, Tari: 20.0μs, BLF: 160kHz, Miller=8
PROFILE_1	E710, E510	PR_ASK, PIE: 2.0, Tari: 7.5μs, BLF: 640kHz, Miller=2
PROFILE_15	E710, E510	PR_ASK, PIE: 2.0, Tari: 7.5μs, BLF: 640kHz, Miller=4
PROFILE_51	E710, E510	DSB, PIE: 1.5, Tari: 6.25μs, BLF: 640, Miller=2
PROFILE_53	E710, E510	PR_ASK, PIE: 1.5, Tari: 7.5,

		BLF:640kHz, Miller=4
PROFILE_11	E710	PR_ASK, PIE: 2.0, Tari: 7.5μs, BLF: 640kHz, FM0
PROFILE_50	E710	DSB, PIE: 1.5, Tari: 6.25μs, BLF: 640kHz, FM0
PROFILE_52	E710	PR_ASK, PIE: 2.0, Tari: 15.0μs, BLF: 426kHz, FM0

2.5.4 getProfile【设置 BLF、Miller、Tari、PIE 】

定义	RfidProfile getProfile();		
说明	获取当前使用的链路组合：BLF(Backscatter Link Frequency)、Miller、Tari、PIE		
参数	名称	类型	备注
返回（RfidProfile）			
参考代码	RfidProfile rfidProfile = RFIDSDKManager.getInstance().getRfidManager().getProfile();		

2.5.5 setInventoryWithSession【设置 Session】

定义	int setInventoryWithSession(int session);
描述	设置盘点 session 注意： 如果处于非自定义模式(InventorySceneMode.CUSTOM_MODE),该参数不可设置; 如果要设置，请先通过 2.3.1 setInventorySceneMode()设置盘点模式为自定义模式(InventorySceneMode.CUSTOM_MODE)。

参数	名称	类型	备注
	session	Session	Session 默认 SESSION.S1; - SESSION.S0 : 0; - SESSION.S1 : 1; - SESSION.S2 : 2; - SESSION.S3 : 3; SESSION.S0:用于查询单个或少量的标签，在相同的时间范围内实现更高的每个标签的读取速率。 SESSION.S1:针对大量标签群体的动态环境进行了优化，尽可能的寻找新标签。 SESSION.S2/SESSION.S3:专为利用持久会话状态的静态标签清单而设计。
返 回 值（int）	成功: RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	RFIDSDKManager.getInstance().getRfidManager().setInventoryWithSession(SESSION.S1);		

2.5.6 getInventoryWithSession【获取 Session】

定义	int getInventoryWithSession();		
描述	获取盘点 session。		
参数	名称	类型	备注
返 回 值（int）	成功： - SESSION.S0 : 0; - SESSION.S1 : 1; - SESSION.S2 : 2; - SESSION.S3 : 3; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	int sesion =		

	RFIDSDKManager.getInstance().getRfidManager().getInventoryWithSession();

2.5.7 setInventoryWithTarget【设置盘点时使用的 Target】

定义	int setInventoryWithTarget(int target);		
描述	设置筛选响应标签的库存标志（Inventoried Flag）状态的 Target，默认:Target.TARGET_AB; 注意： 如果处于非自定义模式(InventorySceneMode.CUSTOM_MODE),该参数不可设置; 如果要设置，请先通过 2.3.1 setInventorySceneMode()设置盘点模式为自定义模式(InventorySceneMode.CUSTOM_MODE)。		
参数	名称	类型	备注
	target	Target	- Target.TARGET_A - 0 : 仅 A 状态 标签响应盘点（Inventoried Flag = A） - Target.TARGET_B - 1 : 仅 B 状态 标签响应盘点（Inventoried Flag = B） - Target.TARGET_AB - 2 : 动态切换状态： 首次查询：A 状态标签响应 后续查询：自动切换为 B 状态标签响应
返回值（int）	成功: RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码			

2.5.8 getInventoryWithTarget【获取盘点会话目标值】

定义	int getInventoryWithTarget();		
描述	获取盘点 Target 值。		
参数	名称	类型	备注
返回值（int）	成功： 0 -TARGET_A; 1 -TARGET_B; 2 -TARGET_AB . 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码			

2.5.9 setInventoryWithStartQvalue【设置盘点时的初始 Q-value】

定义	int setInventoryWithStartQvalue(int QvalueStart);		
描述	设置盘点时的初始 Q-value。 注意： 如果处于非自定义模式(InventorySceneMode.CUSTOM_MODE),该参数不可设置; 如果要设置，请先通过 2.3.1 setInventorySceneMode()设置盘点模式为自定义模式(InventorySceneMode.CUSTOM_MODE)。		
参数	名称	类型	备注
	QvalueStart	Int	起始 Q-value ，决定首次盘点周期（Inventory Round）的时隙数量，后续会自动根据扫描到的标签情况进行动态变化。值的范围 0~15，默认 6;

			标签密度小可以减小 Q，标签密度高可以加大。 备注：Q 为动态 Q，盘点过程会根据标签密度会自动加大或减少 Q 值。
返 回 值（int）	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码			

2.5.10 getInventoryWithStartQvalue【获取盘点的初始 Q-value】

定义	int getInventoryWithStartQvalue();		
描述	获取盘点时的初始 Q-value		
参数	名称	类型	备注
返 回 值（int）	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码			

2.5.11 setInventoryWithPassword【设置盘点时的询查密码】

定义	int setInventoryWithPassword(String password);		
描述	设置盘点的询查密码 。默认是“00000000”		
参数	名称	类型	备注
	password	String	盘点时的询查密码 2 个字长， 16 进制字符串格式。
返 回 值（int）	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码			

2.5.12 getInventoryWithPasssword【获取盘点时的询查密码】

定义	String getInventoryWithPassword();		
描述	获取盘点时的询查密码。		
参数	名称	类型	备注
返回值（String）	返回盘点时的询查密码。		
参考代码			

2.5.13 setInventoryRssiLimit【RSSI 过滤】

定义	int setInventoryRssiLimit(int rssi);		
描述	过滤掉小于设置的 RSSI 值的标签，一般这种场景下，可以同步调整减小功率（功率调整到 20 以下）来达到效果； 使用时请先判断设备是否支持，判断方法： 2.5.12 isSupportInventoryRssiLimit() 仅对于盘点有效，如果要取消过滤限制，请设置 rssi 为 0。 注意： 如果处于非自定义模式(InventorySceneMode.CUSTOM_MODE),该参数不可设置; 如果要设置，请先通过 2.3.1 setInventorySceneMode()设置盘点模式为自定义模式(InventorySceneMode.CUSTOM_MODE)。		
参数	名称	类型	备注
	rssi	int	过滤小于 RSSI 值的标签

返 回 值（int）	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。
参考代码	

2.5.14 isSupportInventoryRssiLimit【是否支持限制 RSSI】

定义	int isSupportInventoryRssiLimit();		
描述	判断盘点时是否支持 RSSI 限制。		
参数	名称	类型	备注
返 回 值（int）	支持：true； 不支持：false。		
参考代码			

2.5.15 setRssiInDbm【设置盘点返回信号的类型】

定义	int setRssiInDbm(boolean isDBm);		
描述	默认是 false. 盘点结果回调方法 onInventoryTag(String EPC, String Data, int rssi); 中的 rssi 默认是大于 0 的的正数信号值，如果要默认输出 dBm (分贝毫瓦) 射频信号强度的绝对测量单位，可以通过该方法设置成 true,		
参数	名称	类型	备注
	isDBm	boolean	true:设置返回 dbm 射频信号强度的绝对测量单位的信号值； false:设置返回大于 0 的正数信号值。
返 回 值（void）			
参考代码			

2.5.16 addMask【添加掩码】

<table>
<tr><td>定义</td><td colspan="3">void addMask(int memMask, int startMask, int lenMask, String datasMask);</td></tr>
<tr><td>说明</td><td colspan="3">添加盘点时的过滤参数,添加该掩码参数后，盘点时只会盘点到设置了对应掩码的标签。过滤掉不属于掩码内的标签。
可以多次添加不同的掩码数据。</td></tr>
<tr><td rowspan="3">参数</td><td>名称</td><td>类型</td><td>备注</td></tr>
<tr><td>memMask</td><td>int</td><td>过滤区域：
TAG 存储区：
MemoryBank.RESERVED(RESERVED)
MemoryBank.EPC (EPC)
MemoryBank.TID (TID)
MemoryBank.USER (USER)</td></tr>
<tr><td>startMask</td><td>int</td><td>起始地址，这里地址的参数跟其他方法参数有差异,数据起始位是从 16 进制字符串的字符所在位置算起：
比如要掩码 EPC 的数据：
EPC :E280FE9889895674
如果要掩码其中的“FE98”那么这里的起始位置则填 12 (因为 EPC 前面还有 CRC+PC)

如果是 TID 或 USER：
USER(或 TID)：0000123456789876 2345
如果要掩码其中的“56789876”那么这里的起始位置填 8</td></tr>
</table>

	lenMask	int	掩码长度，这里长度的参数跟其他方法参数有差异，是需要过滤的数据的 16 进制字符串字符长度，且长度值需是偶数: 比如要掩码 EPC 的数据： EPC :E280FE9889895674 如果要掩码其中的“FE98”那么这里的长度填 4 又或者 EPC：00001234567898762345 如果要掩码其中的“567898762345”那么这里的长度填 12
	datasMask	String	要过滤的内容，十六进制数据。（ 数据长度需是十六进制字符串 2 个字符的倍数）
返回 Void			
参考代码	例子: 标签内容 : Epc:123456789 Tid:234567891 如果要过滤从第二个字母开始的 TID 区域，则需要使用 4567 过滤数据。 您应该设置: memMask:MemoryBank.TID,startMask:2,lenMask：4，datasMask:4567. The code: RFIDSDKManager.getInstance().getRfidManager().addMask(MemoryBank.TID,2,4,”4567”)		

2.5.17 clearMask【清除掩码】

定义	void clearMask();		
说明	清除掩码（过滤）数据		
参数	名称	类型	备注
Void			
参考代码	无		

2.6 标签操作

在 Gen2 协议中:

1 字(Word) = 16 bits (2 字节即 4 个十六进制字符)

示例:

类型	合法输入示例	说明
示例 1	`"E280"`	1 字长= 4 个十六进制字符
示例 2	`"E2808080"`	2 字长 = 8 个十六进制字符
示例 3	`"E2808080A001"`	3 字长 = 12 个十六进制字符

2.6.1 readTag【读取标签】

<table>
<tr><td>定义</td><td colspan="3">TagResult readTag(String EPC,int memBank, int startAdd, int wordCnt, String password);</td></tr>
<tr><td>说明</td><td colspan="3">读取标签信息。</td></tr>
<tr><td rowspan="6">参数</td><td>名称</td><td>类型</td><td>备注</td></tr>
<tr><td>EPC</td><td>Stirng</td><td>标签的 EPC(不包含 CRC 和 PC)，数据长度必须是 1 字长(Word) 的整数倍（即十六进制字符串长度为 4 的倍数）。</td></tr>
<tr><td>password</td><td>String</td><td>标签的访问密码，2 个字长，16 进制字符串格式。</td></tr>
<tr><td>memBank</td><td>int</td><td>TAG 存储区：
MemoryBank.RESERVED(密码区，前 2 个字是销毁密码，后 2 个字是访问密码)
MemoryBank.EPC (EPC)
MemoryBank.TID (TID)
MemoryBank.USER (USER)</td></tr>
<tr><td>startAdd</td><td>int</td><td>要读取/写入的内存区域的起始位置，以字（Word）地址（Word Address） 为单位。</td></tr>
<tr><td>wordCnt</td><td>int</td><td>读取的字(Word)长度（如果要读取的内容很长，建议循环读取,每次读取的最大长度为 32 个字长）</td></tr>
<tr><td>返回(TagResult)</td><td colspan="3">TagResult 类字段信息：
code：状态码：
成功：RfidErrorConstants.SUCCESS；
失败：其他(错误码信息详见:<附录 1：常见错误码>)。
message:错误信息。
data:如果 code == RfidErrorConstants.SUCCESS ，该字段返回读到的标签值内容。</td></tr>
</table>

参考代码	无

2.6.2 writeTag【写标签】

定义	int writeTag(String EPC, String password, int memBank, int startAdd , String writeData);		
说明	写标签。		
参数	名称	类型	备注
	EPC	String	标签的 EPC(不包含 CRC 和 PC)，数据长度必须是 1 字长(Word) 的整数倍（即十六进制字符串长度为 4 的倍数）。
	password	String	标签的访问密码，2 个字长，16 进制字符串格式。
	memBank	int	TAG 存储区： MemoryBank.RESERVED(RESERVED) MemoryBank.EPC (EPC) MemoryBank.TID (TID) MemoryBank.USER (USER)
	startAdd	int	要读取/写入的内存区域的起始位置，以字（Word）地址（Word Address） 为单位。 注意，如果是写 EPC 区： 1>.如果要改变 EPC 的长度，需从 PC 位(起始地址为 1) 开始写，同时要计算 PC 位(PC: Protocol Control, 协议控制位)的值作为写入内容放到写入内容的第一个字； 2>.如果写之后不需要改变 EPC 的长度，

			从起始地址 2 开始写即可。
	writeData	String	待写入的数据，16 进制字符串，数据长度必须是 1 字长(Word) 的整数倍（即十六进制字符串长度为 4 的倍数）。 如果写入内容很大，建议循环写入,每次写入的最大长度为 32 个字长。
返回 (int)	成功：RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

2.6.3 writeTagEpc【写标签 EPC 区】

定义	int writeTagEpc(String EPC, String password, String writeData);		
说明	修改标签的 EPC 值(会默认改动 PC 值)，通过原 EPC 值改写成写入的 EPC 值，即把标签 EPC 号改成新写入的。		
参数	名称	类型	备注
	EPC	String	标签的 EPC(不包含 CRC 和 PC)，数据长度必须是 1 字长(Word) 的整数倍（即十六进制字符串长度为 4 的倍数）。
	password	String	标签的访问密码，2 个字长，16 进制字符串格式。
	writeData	String	待写入的数据，16 进制字符串，数据长度必须是 1 字长(Word) 的整数倍（即十六进制字符串长度为 4 的倍数）。
返回 (int)	成功：RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

2.6.4 writeEpc【随机改写一张标签的 EPC】

定义 1	int writeEpc(String EPC, String password);		
说明	随机改写一张标签的 EPC, 会同时改变其 PC 值		
参数	名称	类型	备注
	EPC	String	16 进制字符串，数据长度必须是 1 字长(Word)的整数倍（即十六进制字符串长度为 4 的倍数）。
	password	String	标签的访问密码，2 个字长，16 进制字符串格式。
返回(int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

2.6.5 lockTag【锁定标签】

定义	int lockTag(String EPC, String password, int lockBank, int lockType);		
说明	锁定标签。		
参数	名称	类型	备注
	EPC	String	标签的 EPC(不包含 CRC 和 PC)，数据长度必须是 1 字长(Word) 的整数倍（即十六进制字符串长

			度为 4 的倍数）。
	password	String	标签的访问密码
	lockBank	int	LockMemBank.KILL_PW - 控制 Kill 密码读写保护设定。 LockMemBank.ACCESS_PW - 控制访问密码读写保护设定。 LockMemBank.EPC - 控制 EPC 存储器读写保护设定。 LockMemBank.TID - 控制 TID 存储器读写保护设定。 LockMemBank.USER - 控制用户存储器读写保护设定。
	lockType	int	可设置的值： LockType.SET_READABLE_OR_WRITABLE LockType.SET_ALWAYS_READABLE_OR_WRITABLE LockType.SET_WITH_PASSWORD_READABLE_OR_WRITABLE LockType.SET_NEVER_READABLE_OR_WRITABLE 该参数在 lockBank 不同时，其设置的含义不一样，具体说明见下表：《**lockBank 和 lockType 的关系说明**》
返回 (int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

lockBank 和 lockType 的关系说明：

	LockMemBank.KILL_PW LockMemBank.ACCESS_PW	LockMemBank.EPC LockMemBank.TID

		LockMemBank.USER
LockType.SET_READABLE_OR_WRITABLE	设置为可读写	设置为可写
LockType.SET_ALWAYS_READABLE_OR_WRITABLE	设置为永远可读写	设置为永远可写
LockType.SET_WITH_PASSWORD_READABLE_OR_WRITABLE	设置为带密码可读写	设置为带密码可写
LockType.SET_NEVER_READABLE_OR_WRITABLE	设置为永远不可读写	设置为永远不可写。

2.6.6 killTag【销毁标签】

定义	int killTag(String EPC, String password);		
说明	销毁标签（销毁密码不能为默认密码 00000000，如果销毁密码是默认密码 00000000,需要先修改成其他，再去销毁）。		
参数	名称	类型	备注
	EPC	String	标签的 EPC(不包含 CRC 和 PC)，数据长度必须是 1 字长(Word) 的整数倍（即十六进制字符串长度为 4 的倍数）。
	password	String	销毁密码,2 个字长，16 进制字符串格式。
返回 (int)	成功：RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

2.6.7 readDataByTid【通过 tid 读取数据】

定义	String readDataByTid(String TID, int memBank, int startAdd , int wordCnt , String password);		
说明	通过标签的 tid 来读取数据		
参数	名称	类型	备注
	TID	String	标签的 TID，十六进制字符串。
	menBank	int	TAG 存储区： MemoryBank.RESERVED(RESERVED) MemoryBank.EPC (EPC) MemoryBank.TID (TID) MemoryBank.USER (USER)
	startAdd	int	要读取/写入的内存区域的起始位置，以字（Word）地址（Word Address）为单位。
	wordCnt	int	读取的字(Word)长度（如果要读取的内容很长，建议每次读取最大 32 个字长的内容，做循环读取）
	password	String	标签的访问密码，2 个字长，16 进制字符串格式。
返回 String	成功返回读取的内容，不成功返回 null。		
参考代码	无		

2.6.8 writeTagByTid【通过 tid 向各存储区写数据】

定义	int writeTagByTid(String TID, int memBank, int startAdd , String password, String writeData);		
说明	通过 TID 写标签数据.		
参数	名称	类型	备注
	TID	String	标签的 TID，16 进制字符串
	memBank	int	TAG 存储区： MemoryBank.RESERVED(RESERVED) MemoryBank.EPC (EPC) MemoryBank.TID (TID) MemoryBank.USER (USER)
	startAdd	int	要读取/写入的内存区域的起始位置，以字（Word）地址（Word Address） 为单位。
	password	String	标签的访问密码，2 个字长，16 进制字符串格式。
	writeData	String	待写入的数据，16 进制字符串，数据长度必须是 1 字长(Word) 的整数倍（即十六进制字符串长度为 4 的倍数）。
返回 (int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

2.6.9 lockByTID【通过 TID 锁定标签】

定义	int lockByTID(String TID, int lockBank, int lockType, String password);		
说明	用于锁定标签。		
参数	名称	类型	备注
	TID	String	标签的 TID，16 进制字符串格式。

	lockBank	int	LockMemBank.KILL_PW - 控制 Kill 密码读写保护设定。 LockMemBank.ACCESS_PW - 控制访问密码读写保护设定。 LockMemBank.EPC - 控制 EPC 存储器读写保护设定。 LockMemBank.TID - 控制 TID 存储器读写保护设定。 LockMemBank.USER - 控制用户存储器读写保护设定。
	lockType	int	可设置的值： LockType.SET_READABLE_OR_WRITABLE LockType.SET_ALWAYS_READABLE_OR_WRITABLE LockType.SET_WITH_PASSWORD_READABLE_OR_WRITABLE LockType.SET_NEVER_READABLE_OR_WRITABLE 该参数在 lockBank 不同时，其设置的含义不一样，具体说明见下表：《**lockBank 和 lockType 的关系说明**》
	password	String	标签的访问密码，2 个字长。
返回 (int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:**<附录 1：常见错误码>**)。		
参考代码	无		

lockBank 和 lockType 的关系说明：

	LockMemBank.KILL_PW LockMemBank.ACCESS_PW	LockMemBank.EPC LockMemBank.TID LockMemBank.USER

LockType.SET_READABLE_OR_WRITABLE	设置为可读写	设置为可写
LockType.SET_ALWAYS_READABLE_OR_WRITABLE	设置为永远可读写	设置为永远可写
LockType.SET_WITH_PASSWORD_READABLE_OR_WRITABLE	设置为带密码可读写	设置为带密码可写
LockType.SET_PERMANENT_NOT_READABLE_OR_WRITABLE	设置为永远不可读写	设置为永远不可写。

2.6.10 killTagByTid【通过 TID 销毁标签】

定义	int killTagByTid(String TID, String password);		
说明	通过 TID 销毁标签（销毁密码不能为默认密码 00000000，如果销毁密码是默认密码 00000000,需要先修改成其他，再去销毁）。		
参数	名称	类型	备注
	TID	String	标签的 TID，16 进制字符串格式。
	password	String	销毁密码,2 个字长，16 进制字符串格式。
返回 (int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

2.6.11 writeTagEpcByTid【通过 TID 写 EPC】

定义	int writeTagEpcByTid(String TID, String password, String writeData);
说明	修改标签的 EPC 值，通过 TID 值,把标签当前的 EPC 值改写成写入的

	(writeData)EPC 值，会自动改写 PC 值。		
参数	名称	类型	备注
	TID	String	标签的 TID。
	password	String	标签的访问密码，2 个字长，16 进制字符串格式。
	writeData	String	待写入的数据，16 进制字符串，数据长度必须是 1 字长(Word) 的整数倍（即十六进制字符串长度为 4 的倍数）。
返回 (int)	成功：RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

2.6.12 maskReadTag【带掩码读取标签】

定义	TagResult maskReadTag(int memBank, int startAdd , int wordCnt , String password, int memMask, int startMask, int lenMask, String datasMask);		
说明	带掩码读取标签内容		
参数	名称	类型	备注
	memBank	int	要读取的 TAG 存储区： MemoryBank.RESERVED(RESERVED) MemoryBank.EPC (EPC) MemoryBank.TID (TID) MemoryBank.USER (USER)
	startAdd	int	要读取/写入的内存区域的起始位置，以字（Word）地址（Word Address） 为单位。
	wordCnt	int	读取的字(Word)长度（如果要读取的内容很长，建议每次读取最大 32 个字长的内容，做循环读取）

	password	String	标签的访问密码，2 个字长，16 进制字符串格式。
	memMask	int	过滤区域： MemoryBank.RESERVED(RESERVED) MemoryBank.EPC (EPC) MemoryBank.TID (TID) MemoryBank.USER (USER)
	startMask	int	起始地址，这里地址的参数跟其他方法参数有差异，数据起始位是从 16 进制字符串的字符所在位置算起： 比如要掩码 EPC 的数据： EPC :E280FE9889895674 如果要掩码其中的“FE98”那么这里的起始位置则填 12 (因为 EPC 前面还有前面还有 CRC+PC) 如果是 TID 或 USER： USER(或 TID)：00001234567898762345 如果要掩码其中的“56789876”那么这里的起始位置填 8
	lenMask	int	掩码长度，这里长度的参数跟其他方法参数有差异（一个长度为 4bits）； 比如要掩码 EPC 的数据： EPC :E280FE9889895674 如果要掩码其中的“FE98”那么这里的长度填 4 或者 EPC：00001234567898762345 如果要掩码其中的“567898762345”那么这里的长度填 12

<table>
<tr><td></td><td></td><td></td><td></td></tr>
<tr><td></td><td>datasMask</td><td>String</td><td>要过滤的内容，十六进制数据。（ 数据长度需是十六进制字符串 2 个字符的倍数）</td></tr>
<tr><td>返回
(String)</td><td colspan="3">TagResult 类字段信息：
code：状态码：
成功：RfidErrorConstants.SUCCESS；
失败：其他(错误码信息详见:<附录 1：常见错误码>)。
message:错误信息。
data:如果 code == RfidErrorConstants.SUCCESS ，该字段返回读到的标签值内容。</td></tr>
<tr><td>参考代码</td><td colspan="3">例子:
标签内容 ：
Epc:123456789999
Tid:23456789000055557777
User:345678912

示例 1：
操作描述：
读取 EPC 区数据（从第 2 字开始，读 3 字长），同时过滤 TID 区（从第 5 字符开始，匹配 2 字符 "56"）
参数解析：
RFIDSDKManager.getInstance().getRfidManager().maskReadTag(
MemoryBank.EPC, // 目标存储区：EPC
2, // 起始字地址：2 (Word Address)
3, // 读取长度：3 Words
"00000000", // 访问密码（默认值）
MemoryBank.TID, // 掩码存储区：TID
4, // 掩码起始位置：第 5 字符（索引 4，0-</td></tr>
</table>

	based） 2, // 掩码长度：2 字符 "56" // 匹配值："56") **示例 2：** **操作描述：** 读取 EPC 区数据（从第 2 字开始，读 3 字长），同时过滤 EPC 区自身（跳过前 8 字符 CRC+PC，从第 13 字符开始匹配 "6789"） **参数解析：** RFIDSDKManager.getInstance().getRfidManager().maskReadTag(MemoryBank.EPC, // 目标存储区：EPC 2, // 起始字地址：2 (Word Address) 3, // 读取长度：3 Words "00000000", // 访问密码（默认值） MemoryBank.EPC, // 掩码存储区：EPC 自身 13, // 掩码起始位置：第 14 字符（索引 13） 4, // 掩码长度：4 字符 "6789" // 匹配值："6789")

2.6.13 maskWriteTag【带掩码写标签】

定义	int maskWriteTag(String writeData, int memBank, int startAdd ,int wordCnt , String password, int memMask, int startMask, int lenMask, String datasMask);		
说明	写标签。		
参数	名称	类型	备注

	writeData	String	待写入的 16 进制数据。
	memBank	int	TAG 存储区： MemoryBank.RESERVED(RESERVED) MemoryBank.EPC (EPC) MemoryBank.TID (TID) MemoryBank.USER (USER)
	startAdd	int	要读取/写入的内存区域的起始位置，以字（Word）地址（Word Address） 为单位。 注意，如果是写 EPC 区： 1>.如果要改变 EPC 的长度，需从 PC 位(起始地址为 1) 开始写，同时要计算 PC 位的值作为写入内容放到写入内容的第一个字； 2>.如果写之后不需要改变 EPC 的长度，从起始地址 2 开始写即可。
	wordCnt	int	读取的字(Word)长度（如果要读取的内容很长，建议每次读取最大 32 个字长的内容，做循环读取）
	password	String	标签的访问密码，2 个字长，16 进制字符串格式。
	memMask	int	过滤区域： MemoryBank.RESERVED(RESERVED) MemoryBank.EPC (EPC) MemoryBank.TID (TID) MemoryBank.USER (USER)
	startMask	int	掩码起始地址，这里地址的参数跟其他方法参数有差异,数据起始位是从 16 进制字符串的字符所在位置算起：

			比如要掩码 EPC 的数据： EPC :E280FE9889895674 如果要掩码其中的“FE98”那么这里的起始位置则填 12（因为 EPC 前面还有前面还有 CRC+PC） 如果是 TID 或 USER： USER(或 TID)：0000123456789876 2345 如果要掩码其中的“56789876”那么这里的起始位置填 8
	lenMask	int	掩码长度，这里长度的参数跟其他方法参数有差异是过滤数据的 16 进制字符串的字符长度，且长度值需是偶数: 比如要掩码 EPC 的数据： EPC :E280FE9889895674 如果要掩码其中的“FE98”那么这里的长度填 4 又或者 EPC：0000123456789876 2345 如果要掩码其中的“567898762345”那么这里的长度填 12
	datasMask	String	要过滤的内容，十六进制数据。（ 数据长度需是十六进制字符串 2 个字符的倍数）
返回 (int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	标签内容 ：		

EPC:123456789999

TID:23456789000055557778888

USER:34567891

示例 1：写 EPC 区 + 过滤 TID 区

操作需求：

将 EPC 区写入数据 999999999999（12 字符，需 3 字长度）

过滤条件：TID 区从第 4 字符（索引 3） 开始取 2 字符，匹配值 "56"

参数转换：

过滤起始位 startMask=3（索引 3 对应第 4 字符）

过滤长度 lenMask=2

代码实现：

```
RFIDSDKManager.getInstance().getRfidManager().maskWriteTag(
    "999999999999",     // 写入数据
    MemoryBank.EPC,     // 写入存储区
    2,                  // 写入起始地址（2 字=8 字符偏移）
    3,                  // 写入长度（3 字=12 字符）
    "00000000",         // 保留参数（固定值）
    MemoryBank.TID,     // 过滤存储区
    3,                  // 过滤起始索引（字符位置）
    2,                  // 过滤长度（字符数）
    "56"                // 过滤匹配值
);
```

示例 2：写 USER 区 + 过滤 EPC 区

操作需求：

将 USER 区写入数据 55555555（8 字符，需 2 字长度）

过滤条件：EPC 区从第 14 字符（索引 13） 开始取 2 字符，匹配值 "67"

	参数转换： 过滤起始位　startMask=13（索引 13 对应第 14 字符） 过滤长度　lenMask=2 **代码实现：** RFIDSDKManager.getInstance().getRfidManager().maskWriteTag(　　"55555555",　　// 写入数据 　　MemoryBank.USER,　　// 写入存储区 　　0,　　// 写入起始地址（0 字=USER 区起始位置） 　　2,　　// 写入长度（2 字=8 字符） 　　"00000000",　　// 保留参数（固定值） 　　MemoryBank.EPC,　　// 过滤存储区 　　13,　　// 过滤起始索引（字符位置） 　　2,　　// 过滤长度（字符数） 　　"67"　　// 过滤匹配值);

2.6.14 readTagExt【读取大容量标签】

定义	TagResult readTagExt(String EPC, int memBank, int startAdd , int wordCnt , String password);		
说明	读取标签信息。		
参数	名称	类型	备注
	EPC	Stirng	标签的 EPC(不包含 CRC 和 PC)，数据长度必须是 1 字长(Word) 的整数倍（即十六进制字符串长度为 4 的倍数）。
	memBank	int	TAG 存储区：

			MemoryBank.RESERVED(RESERVED) MemoryBank.EPC (EPC) MemoryBank.TID (TID) MemoryBank.USER (USER)
	startAdd	int	要读取/写入的内存区域的起始位置，以字（Word）地址（Word Address） 为单位。 注意，如果是写 EPC 区： 1>.如果要改变 EPC 的长度，需从 PC 位(起始地址为 1) 开始写，同时要计算 PC 位的值作为写入内容放到写入内容的第一个字； 2>.如果写之后不需要改变 EPC 的长度，从起始地址 2 开始写即可。
	wordCnt	int	读取的字(Word)长度（如果要读取的内容很长，建议每次读取最大 32 个字长的内容，做循环读取）
	password	String	标签的访问密码，2 个字长，16 进制字符串格式。
返回 (String)	TagResult 类字段信息： **code**：状态码： 成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:**<附录 1：常见错误码>**)。 **message**:错误信息。 **data**:如果 code == RfidErrorConstants.SUCCESS ，该字段返回读到的标签值内容。		
参考代码	无		

2.6.15 writeTagExt【写大容量标签】

<table>
<tr><td>定义</td><td colspan="3">int writeTagExt(String EPC, String password, int memBank, int startAdd , String writeData);</td></tr>
<tr><td>说明</td><td colspan="3">写大容量标签。</td></tr>
<tr><td rowspan="6">参数</td><td>名称</td><td>类型</td><td>备注</td></tr>
<tr><td>EPC</td><td>String</td><td>标签的 EPC(不包含 CRC 和 PC)，数据长度必须是 1 字长(Word) 的整数倍（即十六进制字符串长度为 4 的倍数）。</td></tr>
<tr><td>password</td><td>String</td><td>标签的访问密码，2 个字长，16 进制字符串格式。</td></tr>
<tr><td>memBank</td><td>int</td><td>TAG 存储区：
MemoryBank.RESERVED(RESERVED)
MemoryBank.EPC (EPC)
MemoryBank.TID (TID)
MemoryBank.USER (USER)</td></tr>
<tr><td>startAdd</td><td>int</td><td>要读取/写入的内存区域的起始位置，以字（Word）地址（Word Address） 为单位。
注意，如果是写 EPC 区：
1>.如果要改变 EPC 的长度，需从 PC 位(起始地址为 1) 开始写，同时要计算 PC 位的值作为写入内容放到写入内容的第一个字；
2>.如果写之后不需要改变 EPC 的长度，从起始地址 2 开始写即可。</td></tr>
<tr><td>writeData</td><td>String</td><td>待写入的数据，16 进制字符串，数据长度必须是 1 字长(Word) 的整数倍（即十六进制字符串长度为 4 的倍数）。</td></tr>
<tr><td>返回</td><td colspan="3">成功: RfidErrorConstants.SUCCESS;</td></tr>
</table>

(int)	失败：其他(错误码信息详见:<附录 1：常见错误码>)。
参考代码	无

2.6.16 eraseTag【擦除标签】

定义	int eraseTag(String EPC, String password,int memBank, int startAdd , int wordCnt);		
说明	擦除标签信息,被擦除的数据块其内容将被设置成 0。		
参数	名称	类型	备注
	EPC	Stirng	标签的 EPC(不包含 CRC 和 PC)，数据长度必须是 1 字长(Word) 的整数倍（即十六进制字符串长度为 4 的倍数）。
	password	String	标签的访问密码，2 个字长，16 进制字符串格式。
	memBank	int	TAG 存储区： MemoryBank.RESERVED(密码区，前 2 个字是销毁密码，后 2 个字是访问密码) MemoryBank.EPC (EPC) MemoryBank.TID (TID) MemoryBank.USER (USER)
	startAdd	int	擦除的起始字地址，以字（Word）地址（Word Address） 为单位。
	wordCnt	int	擦除的字(Word)长度。
返回 (int)	成功: RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

2.6.17 findEpc【查找标签】

定义	Tag6C findEpc(String selectEpc);		
说明	查找标签.		
参数	名称	类型	备注
	selectEpc	String	标签的 epc 号
返回 (int)	返回 Tag6C 不为空则表示读到标签。 通过 Tag6C 的 rssi 字段，来判断信号的强弱，以此来判断是否靠近标签，进行定位判断。		
参考代码	无		

2.6.18 lightUpLedTag【点亮指定的 LED 标签】

定义	int lightUpLedTag(String EPC, String password, int milliseconds);		
说明	对指定的 LED 标签进行点亮		
参数	名称	类型	备注
	EPC	String	标签的 EPC
	password	String	标签的访问密码，2 个字长，16 进制字符串格式。
	milliseconds	int	参数范围 100ms - 25500ms ； 要以 100ms 的倍数传参。 比如需要点亮 最大的 25500 毫秒，传 25500
返回 (int)	成功: RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		

参考代码	无

2.6.19 startInventoryLed【询查 ELD 标签】

定义	int startInventoryLed(int manufacturers, List<String> epcs);		
说明	对选中的 LED 标签进行亮灯		
参数	名称	类型	备注
	manufacturers	int	厂商编号（0 或 1） 0：凯路威 1：宜联
	epcs	List<String>	需要点亮的 EPC 标签集合
返回 (int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

2.6.20 stopInventoryLed【停止询查 ELD 标签】

定义	void stopInventoryLed();

说明	停止 LED 标签亮灯		
参数	名称	类型	备注
返回 (void)			
参考代码	无		

2.7 频段和其他设置

2.7.1 setWorkRegion【设置区域频段】

定义	int setWorkRegion(CountryCode cntyCode);		
说明	根据区域设置频段； 备注：如果要设置的地区，在 附录 4: 区域对照表 中找不到，您可以参考 API 中 2.7.3 setFrequencyRegion() 方法进行设置。		
参数	名称	类型	备注
	cntyCode	CountryCode	国际区域参数（附录 4: 区域对照表） 例如： CountryCode.ALB CountryCode.BEL ...
返回 (int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	int ret = RFIDSDKManager.getInstance().getRfidManager() .setWorkRegion(CountryCode.ALB);		

2.7.2 getWorkRegion【获取工作频率】

定义	CountryCode getWorkRegion();		
说明	获取当前设备使用的区域频段信息。 如果获取结果为 null,则可能是之前没有通过 setWorkRegion(CountryCode cntyCode) 设置过，可以使用 2.7.4 getFrequencyRegion() 方法来获取当前的频段信息。		
参数	名称	类型	备注
返回(CountryCode)	成功：CountryCode 不为 null ； 失败：CountryCode 为 null；		
参考代码			

2.7.3 setFrequencyRegion【设置频率区域】

定义	int setFrequencyRegion(FrequencyRegion region, int btStartIndex, int btEndIndex);		
说明	设置频率区域。		
参数	名称	类型	备注

	region	FrequencyRegion	频段类型。
			FrequencyRegion.UKRAINE;//Ukraine band
			FrequencyRegion.CHINESE1;//Chinese band 1
			FrequencyRegion.EU;//EU band
			FrequencyRegion.CHINESE2;//Chinese band 2
			FrequencyRegion.US;//US band
			FrequencyRegion.KOREAN;//Korean band
			FrequencyRegion.PERU;//Peru band
			FrequencyRegion.US3;//US band3
			FrequencyRegion.Taiwan;//Taiwan band
			
			详见：附录 2：频段说明

	btStartInd	int	频谱的起始频点索引。

	ex		
	btEndIndex	int	频谱的终止频点索引 设置射频输出频谱的范围。 规则： 1、起始频率和终止频率应在规定的规定范围内。 2、起始频率应等于或低于终止频率。 3、结束频率等于起始频率，即使用单频点。
返回 (int)	成功：RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	int ret = RFIDSDKManager.getInstance().getRfidManager().setFrequencyRegion (FrequencyRegion.US, FrequencyRegion.US.getMinChannelIndex(),FrequencyRegion.US.getMaxChannelIndex());		

2.7.4 getFrequencyRegion【获取频率区域】

定义	FrequencyRegion getFrequencyRegion();		
说明	获取频率区域。		
参数	名称	类型	备注
返回 (FrequencyRegion)	FrequencyRegion frequencyRegion = RFIDSDKManager.getInstance().getRfidManager().getFrequencyRegion(); 1. 获取频谱频段：`frequencyRegion.getbtRegion()` 2. 获取起始频点索引：`frequencyRegion.getMinChannelIndex()` 3. 获取终止频点索引：`frequencyRegion.getMaxChannelIndex()`		

	4. 计算起始频率（MHz）： float startFreq = frequencyRegion.getStartFreqMHz() + frequencyRegion.getMinChannelIndex() * frequencyRegion.getStepSizeMHz() 5. 计算结束频率（MHz）： float endFreq = frequencyRegion.getStartFreqMHz() + frequencyRegion.getMaxChannelIndex() * frequencyRegion.getStepSizeMHz() 如果`frequencyRegion`为null，表示获取失败。
参考代码	

2.7.5 getFirmwareVersion【获取固件版本】

定义	String getFirmwareVersion();		
说明	获取RFID模块的固件版本。		
参数	名称	类型	备注
返回(String)	返回String类型的RFID模块固件版本信息		
参考代码	无		

2.7.6 setTagFocus【设置TagFocus命令】

定义	int setTagFocus(int enable);		
说明	该命令用于设置TagFocus标记，仅限Ex10系列。		
参数	名称	类型	备注
	enable	int	0-关闭；1-打开

返回 (int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。
参考代码	无

2.7.7 getTagFocus【读取 TagFocus 命令】

定义	int getTagFocus();		
说明	该命令用于读取TagFocus标记，仅限Ex10系列。		
参数	名称	类型	备注
返回 (int)	成功：RfidErrorConstants.SUCCESS-关闭；1-开启； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

2.7.8 getModuleFirmware【获取模块固件信息】

定义	String getModuleFirmware();		
说明	获取模块固件信息		
参数	名称	类型	备注
返回 (String)	模块信息		
参考代码	无		

2.7.9 GetReaderType【获取模块类型】

定义	int GetReaderType();
说明	获取模块类型

参数	名称	类型	备注
返回 (int)	成功：大于 0 失败：-1。		
参考代码	无		

2.7.10 getReaderTemperature【查询内部温度】

定义	String getReaderTemperature();		
说明	查询内部温度。		
参数	名称	类型	备注
返回 (String)	返回字符串格式的温度值，单位（摄氏度）		
参考代码	无		

2.7.11 setCustomRegion【设置自定义频段】

定义	int setCustomRegion(int flags, int FreSpace, int FreNum, int StartFre);		
说明	设置自定义频段，		
参数	名称	类型	备注
	flags	int	下电是否保存设置标识 0：保存 1：不保存
	int	FreSpace	频段间隔（单位 KHz），参数需要是 125 的倍数。

	int	FreNum	频点数量
	int	StartFre	起始频率（单位 KHz）
返回(int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码			

2.7.12 getCustomRegion【获取自定义频段】

定义	CustomRegionBean getCustomRegion();		
说明	获取自定义频段信息； 注意：设置自定义频段后，调用 getFrequencyRegion()无法获取标准频段的相关信息。		
参数	名称	类型	备注

返回(CustomRegionBean)	CustomRegionBean customRegion = rfidManager.getCustomRegion();
	if (customRegion != null) {
	// 提取参数
	int band = customRegion.band[0]; //频段区域
	int stepKHz = customRegion.FreSpace[0]; //频段间隔（单位 KHz）
	int channelCount = customRegion.FreNum[0]; //频点数量
	int startFreqKHz = customRegion.StartFre[0]; //起始频率（单位 KHz）
	
	} else {
	System.err.println("获取自定义频段配置失败！");
	}		

参考代码	无		

2.7.13 getEx10Version［获取 E 系列芯片信息］

定义	ExVersionInfo getEx10Version();		
说明	获取使用的 RFID 芯片的版本和类型，非模块版本信息。		
参数	名称	类型	备注

返回（ ExVersionInfo ）	RFID 的芯片信息，包含芯片类型和版本号.
	
	ExVersionInfo 说明 :
	public class ExVersionInfo {
	public int code ; //返回值 成功: RfidErrorConstants.SUCCESS 失败：其他
	public String versionInfo ; //芯片的固件版本号
	public int type ; //
	EXChipType.E310: E310;
	EXChipType.E510: E510;
	EXChipType.E710: E710;
	EXChipType.OTHER: 其他
	}

参考代码	无

2.7.14 enableLog【开启或者关闭日志】

定义	void enableLog(boolean enable);		
说明	开启或者关闭日志。		
参数	名称	类型	备注
	enable	boolean	true:开启日志；false:关闭日志
返回 (void)			
参考代码	无		

2.8 蜂鸣器设置

2.8.1 setBeepEnable【设置开启蜂鸣器】

定义	int setBeepEnable(boolean enable);		
说明	开启或关闭蜂鸣器，默认关闭，如果要RFID读到标签后，自动发出声音，需要调用开启。		
参数	名称	类型	备注
	enable	boolean	true-打开,false-关闭
返回(int)	成功：RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

2.8.2 setBeepVolume【设置蜂鸣器音量】

定义	int setBeepVolume(int volume);		
说明	设置蜂鸣器音量,volume 默认为4, 注意：这个值设置越大耗电会越高，使用的时候请根据实际场景进行设置或使用默认。		
参数	名称	类型	备注
	volume	int	范围 1-10，默认 4. 1 最小，10 最大。
返回(int)	成功：RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

2.8.3 startBeep【控制蜂鸣器响一次】

定义	void startBeep();
说明	主动控制蜂鸣发出一次响声.

参数	名称	类型	备注
返回(int)			
参考代码	无		

3 固件升级

注意事项：

1>.RFID 固件的升级：包含 RFID 模块的升级和 RFID 芯片的升级；

2>.固件升级属于高级操作，请谨慎操作；

3>.有需要更新前，请与我司确认升级的必要性已经升级的最新固件版本，防止升级到错误的固件。

使用的类：FirmwareManager.getInstance();

3.1 一体机设备升级(包含 RFG91 UART)

3.1.1 updateReaderFirmwareByFile【RFID 模块升级--通过读取文件】

定义	void updateReaderFirmwareByFile(Context context, FWUpdateCallback myCallback, String binName, String binPath);		
说明	升级模块固件，需要对文件有读写权限，如果文件放在 sdcard 内部存储目录，软件需要有所有文件的管理权限		
参数	名称	类型	备注
	context	Context	上下文
	myCallback	FWUpdate Callback	更新过程回调接口 FWUpdateCallback 说明： public abstract class FWUpdateCallback { /** * 升级结果回调 * @param res * 0：升级成功

			* 11：升级失败 * 12:正在升级中 * 13：进入升级模式失败 * 14：进入升级模式成功 * 20:读取文件出错 */ public abstract void updateResult(int res); /** * 正在升级的时候，会进行回调 * @param progress 升级过程中返回的升级进度百分比 */ public abstract void updateProcess(int progress); }
	binName	String	文件名称(不带后缀格式)， 用于和模块型号对比 ，实际使用时，需要向我司确认名称。
	binPath	String	固件名称的存储位置，应用需要对这个路径有读写权限，路径示例：/storage/emulated/0/RRU7180M_V2.09.0.0.bin
返回(int)			
参考代码	无		

3.1.2 updateReaderFirmwareByByte【RFID 模块升级--通过字节流】

<table>
<tr><td>定义</td><td colspan="3">void updateReaderFirmwareByByte(Context context, FWUpdateCallback myCallback, String binName, byte[] fileBuff);</td></tr>
<tr><td>说明</td><td colspan="3">升级模块固件，需要固件对应的字节流数据，可以从本地文件获取</td></tr>
<tr><td rowspan="3">参数</td><td>名称</td><td>类型</td><td>备注</td></tr>
<tr><td>context</td><td>Context</td><td>上下文</td></tr>
<tr><td>myCallback</td><td>FWUpdate
Callback</td><td>更新过程回调接口
FWUpdateCallback 说明：
public abstract class FWUpdateCallback {
/**
* 升级结果回调
* @param res
* 0：升级成功
* 11：升级失败
* 12:正在升级中
* 13：进入升级模式失败
* 14：进入升级模式成功
* 20:读取文件出错
*/
public abstract void updateResult(int res);

/**
* 正在升级的时候，会进行回调
* @param progress 升级过程中返回的升级进度百分比
*/</td></tr>
</table>

			public abstract void updateProcess(int progress); }
	binName	String	文件名称(不带后缀格式)， 用于和模块型号对比 ，实际使用时，需要向我司确认名称。
	fileBuff	byte[]	固件文件读取到的字节数据流
返回 (int)			
参考代码	无		

3.1.3 updateEx10ChipFirmwareByFile【RFID 芯片升级--通过读取文件】

定义	void updateEx10ChipFirmwareByFile(Context context, FWUpdateCallback myCallback, String binPath);		
说明	升级 RFID 芯片固件，需要对文件有读写权限，如果文件放在 sdcard 内部存储目录，软件需要有所有文件的管理权限		
参数	名称	类型	备注
	context	Context	上下文
	myCallback	FWUpdate Callback	更新过程回调接口 FWUpdateCallback 说明： public abstract class FWUpdateCallback { /** * 升级结果回调 * @param res * 0：升级成功

			* 11：升级失败 * 12:正在升级中 * 13：进入升级模式失败 * 14：进入升级模式成功 * 20:读取文件出错 */ public abstract void updateResult(int res); /** * 正在升级的时候，会进行回调 * @param progress 升级过程中返回的升级进度百分比 */ public abstract void updateProcess(int progress); }
	binPath	String	固件名称的存储位置，应用需要对这个路径有读写权限，路径示例：/storage/emulated/0/ex10_appV2.01.02.bin
返回 (int)			
参考代码	无		

3.1.4 updateEx10ChipFirmwareByByte【RFID 芯片升级--通过字节流】

<table>
<tr><td>定义</td><td colspan="3">void updateEx10ChipFirmwareByByte(Context context, FWUpdateCallback myCallback, byte[] szbuff);</td></tr>
<tr><td>说明</td><td colspan="3">升级 RFID 芯片固件，需要固件对应的字节流数据，可以从本地文件获取</td></tr>
<tr><td rowspan="3">参数</td><td>名称</td><td>类型</td><td>备注</td></tr>
<tr><td>context</td><td>Context</td><td>上下文</td></tr>
<tr><td>myCallback</td><td>FWUpdate
Callback</td><td>更新过程回调接口
FWUpdateCallback 说明：
public abstract class FWUpdateCallback {
/**
* 升级结果回调
* @param res
* 0：升级成功
* 11：升级失败
* 12:正在升级中
* 13：进入升级模式失败
* 14：进入升级模式成功
* 20:读取文件出错
*/
public abstract void updateResult(int res);

/**
* 正在升级的时候，会进行回调
* @param progress 升级过程中返回的升级进度百分比
*/
public abstract void updateProcess(int progress);</td></tr>
</table>

			}
	szbuff	byte[]	芯片固件文件读取出来的字节流数据
返回 (int)			
参考代码	无		

3.2 蓝牙设备升级

3.2.1 updateBLEReaderFirmwareByFile【RFID 模块升级--通过读取文件】

定义	void updateBLEReaderFirmwareByFile(Context context, String address, FWUpdateCallback myCallback, String binName, String binPath);		
说明	升级模块固件，需要对文件有读写权限，如果文件放在 sdcard 内部存储目录，软件需要有所有文件的管理权限		
参数	名称	类型	备注
	context	Context	上下文
	address	String	蓝牙设备的 MAC 地址
	myCallback	FWUpdate Callback	更新过程回调接口 FWUpdateCallback 说明： public abstract class FWUpdateCallback { /** * 升级结果回调

			* @param res * 0：升级成功 * 11：升级失败 * 12:正在升级中 * 13：进入升级模式失败 * 14：进入升级模式成功 * 20:读取文件出错 */ public abstract void updateResult(int res); /** * 正在升级的时候，会进行回调 * @param progress 升级过程中返回的升级进度百分比 */ public abstract void updateProcess(int progress); }
	binName	String	文件名称(不带后缀格式)， 用于和模块型号对比 ，实际使用时，需要向我司确认名称。
	binPath	String	固件名称的存储位置，应用需要对这个路径有读写权限，路径示例：/storage/emulated/0/RRU7180M_V2.09.0.0.bin
返回 (int)			
参考代码	无		

3.2.2 updateBLEReaderFirmwareByByte【RFID 模块升级--通过读取文件】

<table>
<tr><td>定义</td><td colspan="3">void updateBLEReaderFirmwareByByte(Context context, String address, FWUpdateCallback myCallback, String binName, byte[] fileBuff);</td></tr>
<tr><td>说明</td><td colspan="3">升级模块固件，需要固件对应的字节流数据，可以从本地文件获取</td></tr>
<tr><td rowspan="4">参数</td><td>名称</td><td>类型</td><td>备注</td></tr>
<tr><td>context</td><td>Context</td><td>上下文</td></tr>
<tr><td>address</td><td>String</td><td>蓝牙设备的 MAC 地址</td></tr>
<tr><td>myCallback</td><td>FWUpdate
Callback</td><td>更新过程回调接口
FWUpdateCallback 说明：
public abstract class FWUpdateCallback {
/**
* 升级结果回调
* @param res
* 0：升级成功
* 11：升级失败
* 12:正在升级中
* 13：进入升级模式失败
* 14：进入升级模式成功
* 20:读取文件出错
*/
public abstract void updateResult(int res);

/**
* 正在升级的时候，会进行回调</td></tr>
</table>

			* @param progress 升级过程中返回的升级进度百分比 */ public abstract void updateProcess(int progress); }
	binName	String	文件名称(不带后缀格式)， 用于和模块型号对比 ，实际使用时，需要向我司确认名称。
	fileBuff	byte[]	固件文件读取出来的字节流数据
返回 (int)			
参考代码	无		

3.2.3 updateBLEEx10ChipFirmwareByFile【RFID 模块升级--通过字节流】

定义	void updateBLEEx10ChipFirmwareByFile(Context context, String address, FWUpdateCallback myCallback, String binName, String binPath);		
说明	升级 RFID 芯片固件，需要对文件有读写权限，如果文件放在 sdcard 内部存储目录，软件需要有所有文件的管理权限		
参数	名称	类型	备注
	context	Context	上下文
	address	String	蓝牙设备的 MAC 地址
	myCallback	FWUpdate Callback	更新过程回调接口 FWUpdateCallback 说明： public abstract class FWUpdateCallback {

			/** * 升级结果回调 * @param res * 0：升级成功 * 11：升级失败 * 12:正在升级中 * 13：进入升级模式失败 * 14：进入升级模式成功 * 20:读取文件出错 */ public abstract void updateResult(int res); /** * 正在升级的时候，会进行回调 * @param progress 升级过程中返回的升级进度百分比 */ public abstract void updateProcess(int progress); }
	binName	String	文件名称(不带后缀格式)， 用于和模块型号对比 ，实际使用时，需要向我司确认名称。
	binPath	String	芯片固件名称的存储位置，需要传绝对路径，例如：/storage/emulated/0/ex10_appV2.01.02.bin
返回			

(int)	
参考代码	无

3.2.4 updateBLEEx10ChipFirmwareByByte【RFID 模块升级--通过字节流】

定义	void updateBLEEx10ChipFirmwareByByte(Context context, String address, FWUpdateCallback myCallback, String binName, byte[] fileBuff);		
说明	升级 RFID 芯片固件，需要固件对应的字节流数据，可以从本地文件获取		
参数	名称	类型	备注
	context	Context	上下文
	address	String	蓝牙设备的 MAC 地址
	myCallback	FWUpdate Callback	更新过程回调接口 FWUpdateCallback 说明： public abstract class FWUpdateCallback { /** * 升级结果回调 * @param res * 0：升级成功 * 11：升级失败 * 12:正在升级中 * 13：进入升级模式失败 * 14：进入升级模式成功 * 20:读取文件出错 */ public abstract void updateResult(int res);

			/** * 正在升级的时候，会进行回调 * @param progress 升级过程中返回的升级进度百分比 */ public abstract void updateProcess(int progress); }
	binName	String	文件名称(不带后缀格式)， 用于和模块型号对比 ，实际使用时，需要向我司确认名称。
	fileBuff	byte[]	固件文件读取到的字节数据流
返回 (int)			
参考代码	无		

4. 蓝牙、UART 分体设备接口说明(非一体设备)

适用设备：本章节只适用于 RFG91 UART 分体设备和蓝牙设备；

功能：获取条码扫描结果、设备状态回调监听、设备信息获取、参数设置等。

使用条件：接口的正常运行依赖于上面［2.1 初始化和连接］章节的初始化成功为前提。

使用的类：GripDeviceManager.getInstance();

4.1 扫描条码的结果获取

接收扫描条码的结果。

registerBarcodeCallback【注册扫描头扫描条码的回调】

定义	void registerBarcodeCallback(BarcodeCallback cb);		
说明	注册扫条码的回调。		
参数	名称	类型	备注
	cb	BarcodeCallback	回调
返回 (void)	无		

参考代码	GripDeviceManager.getInstance().registerBarcodeCallback(new
	BarcodeCallback() {
	@Override
	public void onResult(byte[] barcode) {
	//barcode 扫描到的条码结果
	}
	});		

4.2 设备状态的回调监听

4.2.1 setKeyEventListener【注册按键事件监听】

<table>
<tr><td>定义</td><td colspan="3">void setKeyEventListener(KeyEventListener cb);</td></tr>
<tr><td>说明</td><td colspan="3">注册回调（该方法会在按下或抬起扫描按键时，监听到相应的事件）。
RFG91 按下按键后，默认不会执行 RFID 的相关操作，如果要通过 RFG91 上的按键来触发 RFID 的动作，需要监听本回调事件，并进行相应的接口调用，来触发 RFID。</td></tr>
<tr><td rowspan="2">参数</td><td>名称</td><td>类型</td><td>备注</td></tr>
<tr><td>cb</td><td>KeyEventListener</td><td>按键状态回调：
public void event(int keyCode, boolean isDown)
{
//keyCode== BTKeyEvent.BT_SCAN The handle button was pressed
//isDown true: down false: up
}</td></tr>
<tr><td>返回
(void)</td><td colspan="3">无</td></tr>
<tr><td>参考代码</td><td colspan="3">GripDeviceManager.getInstance().setKeyEventListener(new KeyEventListener()
{
@Override
public void event(int keyCode, boolean isDown) {
//keyCode== BTKeyEvent.BT_SCAN The handle button was pressed
//isDown true: down false: up
if (keyCode == BTKeyEvent.BT_SCAN) {</td></tr>
</table>

	if (isDown) {
	//start Inventory
	//RFIDSDKManager.getInstance().getRfidManager().startInventory(1);
	} else {
	//stop Inventory
	//RFIDSDKManager.getInstance().getRfidManager().stopInventory();
	}
	}
	}
	});

4.2.2 setBatteryGripListener【注册设备充电状态、电池电量变化的监听】

定义	void setBatteryGripListener(BatteryGripListener cb);		
说明	注册设备电池相关状态的回调，该方法会设备充电状态改变，电池电量变化时，监听到相应的值的改变），因为该方法是在状态改变的时候才会回调，所以如果要立刻获取充电状态以及设备电量，需要通过 4.5 getIsChange 和 4.4 getElectricQuantity 方法来获取。		
参数	名称	类型	备注

	cb	BatteryGripListener	设备充电状态、电量信息回调：
			new BatteryGripListener() {
			@Override
			public void isChange(int change) {
			//change 0：未充电 1：充电
			}
			@Override
			public void level(int percent) {

			//percent 电量百分比 0-100%
			}
			};

返回 (void)	无		

参考代码	GripDeviceManager.getInstance().setBatteryGripListener(new
	BatteryGripListener() {
	@Override
	public void isChange(int change) {
	//change 0：未充电 1：充电
	}
	@Override
	public void level(int percent) {
	//percent 电量百分比 0-100%
	}
	});		

4.3 setSwitchCallback【监听 RFID 模块之间的切换】

定义	void setSwitchCallback(ModelInfoCallback cb);		
说明	使用类：RFIDSDKManager.getInstance(); 监听 DT610 自带 RFID 和 RFG91 UART 设备的切换使用时，切换后 RFID 模块状态的回调。 注意：只在 DT610 和 RFG91 UART 设备搭配使用时才有效，不是该组合下的设备调用该方法无效。		
参数	名称	类型	备注
	cb	ModelInfoCallback	切换后的模块信息回调： interface ModelInfoCallback {

			/**
			* @param type 0 : 自带模块（DT610 自带模块） 1: UART 模块 （RFG91 UART 设备）
			* @param firmware 模块版本号
			* @param power 使用的功率值
			*/
			void onInfo(int type, String firmware, int power);
			
			/**
			* @param status
			* ModelStatus.Connected = 0; //合上
			* ModelStatus.Separate = 1; //分离
			* ModelStatus.StartingUp = 2; //待机
			* ModelStatus.Shutdown = 3; //关机
			*/
			void onModuleStatus(int status);
			
			}

返回 (void)	无		

参考代码	RFIDSDKManager.getInstance().setSwitchCallback(new ModelInfoCallback() {
	@Override
	public void onModuleStatus(int status) {
	}
	@Override
	public void onInfo(int type, String firmware, int power) {		

	} });

4.3 startScanBarcode【开始扫描条码】

定义	int startScanBarcode(boolean enable);		
说明	开始扫描标签。		
参数	名称	类型	备注
	enable	boolean	true:触发开始扫码 false:触发停止扫码
返回 (int)	成功：RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	无		

4.4 getElectricQuantity【获取电池电量】

定义	int getElectricQuantity();		
说明	获取电池电量		
参数	名称	类型	备注
返回 (int)	成功：大于等于 0; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	int value = GripDeviceManager.getInstance().getElectricQuantity();		

4.5 getIsChange【获取电池充电状态】

定义	int getIsChange();

说明	获取设备充电状态。		
参数	名称	类型	备注
返回 (String)	成功：RfidErrorConstants.SUCCESS：未在充电　1：正在充电； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	int value = GripDeviceManager.getInstance().getIsChange();		

4.6 getVersionSystem【获取设备版本号】

定义	String getVersionSystem();		
说明	获取设备的版本号		
参数	名称	类型	备注
返回 String	成功返回读取的内容，不成功返回 null。		
参考代码	String sysVer = GripDeviceManager.getInstance().getVersionSystem();		

4.7 getVersionBLE【获取蓝牙版本号】

定义	String getVersionBLE();		
说明	获取设备的蓝牙固件版本号		
参数	名称	类型	备注
返回 String	成功返回读取的内容，不成功返回 null。		
参考代码	String bleVer = GripDeviceManager.getInstance().getVersionBLE();		

4.8 getDeviceSN【获取设备 SN 号】

定义	String getDeviceSN();		
说明	获取设备 SN		
参数	名称	类型	备注
返回 String	成功返回读取的内容，不成功返回 null。		
参考代码	String sn= GripDeviceManager.getInstance().getDeviceSN();		

4.9 setBeepRange【设置蜂鸣器音量】

定义	int setBeepRange(int beep);		
说明	设置蜂鸣器音量		
参数	名称	类型	备注
	beep	int	范围：0-10 如果是 0，则关闭蜂鸣器。
返回 (int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	int ret = GripDeviceManager.getInstance().setBeepRange(10);		

4.10 getBeepRange【获取蜂鸣器音量】

定义	int getBeepRange();

说明	获取蜂鸣器音量。		
参数	名称	类型	备注
返回(int)	返回蜂鸣器音量大小(小于 0 则获取失败)		
参考代码	int beepValue = GripDeviceManager.getInstance().getBeepRange();		

4.11 setSleepTime【设置休眠时间】

定义	int setSleepTime(int time);		
说明	设置休眠时间，单位：秒		
参数	名称	类型	备注
	time	int	范围：1-3600
返回(int)	成功：RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	int ret = GripDeviceManager.getInstance().setSleepTime(60);		

4.12 getSleepTime【获取休眠时间】

定义	int getSleepTime();		
说明	获取当前设备的休眠时间。		
参数	名称	类型	备注
返回(int)	返回休眠时间(小于 0 则获取失败)		
参考代码	int sleepValue = GripDeviceManager.getInstance().getSleepTime();		

4.13 setPowerOffTime【设置超时关机时间】

定义	int setPowerOffTime(int offTime);		
说明	设置超时关机的时间,如果没有对设备进行接口调用和按设备上的实体按键，在超时对应时间后，设备将关机。单位：秒		
参数	名称	类型	备注
	beep	int	范围：1-10
返回 (int)	成功: RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	int ret = GripDeviceManager.getInstance().setBeepRange(10);		

4.14 getPowerOffTime【获取蜂鸣器音量】

定义	int getPowerOffTime();		
说明	获取超时关机的时间。		
参数	名称	类型	备注
返回 (int)	返回超时关机的时间(小于 0 则获取失败)		
参考代码	int OFFValue = GripDeviceManager.getInstance().getPowerOffTime();		

4.15 getBLEMac【获取蓝牙设备的 MAC 地址】

定义	String getBLEMac();		
说明	获取蓝牙设备的 MAC 地址,仅限蓝牙设备。		
参数	名称	类型	备注
返回 String	成功：返回 MAC 地址，例如：41:BB:00:F8:DB:51 失败：返回空。		

参考代码	String mac= GripDeviceManager.getInstance().getBLEMac();

4.16 setOfflineModeOpen【设置离线模式】

定义	int setOfflineModeOpen(int mode);		
说明	设置离线模式。		
参数	名称	类型	备注
	mode	int	范围：0-1 0：关闭离线模式 1：开启离线模式
返回 (int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	int ret = GripDeviceManager.getInstance().setOfflineModeOpen(1);		

4.17 getOfflineModeOpen【获取离线模式开启状态】

定义	int getOfflineModeOpen();		
说明	获取离线模式状态。		
参数	名称	类型	备注
返回 (int)	返回离线模式状态 0：离线模式处于关闭状态 1：离线模式处于开启状态		
参考代码	int value = GripDeviceManager.getInstance().getOfflineModeOpen();		

4.18 setOfflineTransferClearData【设置离线模式传输后是否自动清除数据】

定义	int setOfflineTransferClearData(int mode);		
说明	设置离线模式下，数据传输后，是否自动清除设备里的缓存。		
参数	名称	类型	备注
	mode	int	范围：0-1 0：不自动清除 1：自动清除
返回 (int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	int ret = GripDeviceManager.getInstance().setOfflineTransferClearData(1);		

4.19 getOfflineTransferClearData【获取离线模式下自动清除数据的状态】

定义	int getOfflineTransferClearData();		
说明	获取离线数据传输后，是否自动清除数据的模式状态。		
参数	名称	类型	备注
返回 (int)	返回离线模式下数据清除的状态 0：离线数据传输后，不自动清除数据 1：离线数据传输后，自动清除数据		
参考代码	int value = GripDeviceManager.getInstance().getOfflineTransferClearData();		

4.20 setOfflineTransferDelay【设置离线模式下传输数据，数据

之间的发送间隔时间】

定义	int setOfflineTransferDelay(int delay);		
说明	设置离线模式下，数据传输每条数据的间隔时间。		
参数	名称	类型	备注
	delay	int	范围：0-10000ms
返回 (int)	成功：RfidErrorConstants.SUCCESS； 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	int ret = GripDeviceManager.getInstance().setOfflineTransferDelay(0);		

4.21 getOfflineTransferClearData【获取离线模式下自动清除数据的状态】

定义	int getOfflineTransferClearData();		
说明	获取离线数据传输后，是否自动清除数据的模式状态。		
参数	名称	类型	备注
返回 (int)	结果大于等于 0，则成功，单位 ms 小于 0，则获取失败。		
参考代码	int value = GripDeviceManager.getInstance().getOfflineTransferDelay();		

4.22 getOfflineQueryNum【获取离线数据的数量】

定义	int[] getOfflineQueryNum();
说明	获取离线状态下，读取的 RFID 标签和条码数量。

参数	名称	类型	备注
返回 (int[])	返回集合：int[] numArr 如果 numArr 长度大于 0，则表示获取成功， numArr[0]： 缓存的 RFID 数量 numArr[1]： 缓存的扫描条码数量 如果 numArr 等于空或长度等于 0，则获取失败		
参考代码	int[] numArr= GripDeviceManager.getInstance().getOfflineQueryNum();		

4.23 getOfflineQueryMem【获取离线数据占用的内存百分比】

定义	double [] getOfflineQueryMem();		
说明	获取离线状态下，扫描的数据所占用的存储空间占比。		
参数	名称	类型	备注
返回 (int)	返回集合：double[] memArr 如果 memArr 长度大于 0，则表示获取成功， memArr[0]： 缓存的 RFID 占比 memArr[1]： 缓存的扫描条码占比 如果 memArr 等于空或长度等于 0，则获取失败		
参考代码	double[] memArr= GripDeviceManager.getInstance().getOfflineQueryMem();		

4.24 offlineManaulClearData【主动清除离线模式下保存的数据】

定义	int offlineManaulClearData();		
说明	主动清除离线模式下保存的数据。		
参数	名称	类型	备注
返回 (int)	成功：RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	int value = GripDeviceManager.getInstance().offlineManaulClearData();		

4.25 offlineStartTransferRFID【主动清除离线模式下保存的RFID数据】

定义	int offlineStartTransferRFID();		
说明	开始传输离线数据保存的数据。 数据统一通过 RFID 的 IRfidCallback 回调函数回调。		
参数	名称	类型	备注
返回 (int)	成功：RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	int ret= GripDeviceManager.getInstance().offlineStartTransferRFID();		

4.26 offlineStartTransferScan【主动清除离线模式下保存的RFID数据】

定义	int offlineStartTransferScan();		
说明	开始传输离线数据保存的数据。 数据通过 BarcodeCallback 回调函数回调。		
参数	名称	类型	备注
返回 (int)	成功：RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	int ret= GripDeviceManager.getInstance().offlineStartTransferScan();		

4.27 modeResetFactory【设备固件恢复出厂设置】

定义	int modeResetFactory();		
说明	重置设备的固件设置（非蓝牙固件和 RFID 固件）。		
参数	名称	类型	备注
返回 (int)	成功：RfidErrorConstants.SUCCESS; 失败：其他(错误码信息详见:<附录 1：常见错误码>)。		
参考代码	int ret= GripDeviceManager.getInstance().modeResetFactory();		

附录 1：常见错误码

错误代码(RfidErrorConstants)	描述
SUCCESS	执行成功。
TAG_NOT_FOUND	未询查到电子标签。
INSUFFICIENT_POWER	电源不足，标签电源不足，无法执行存储写入操作。
MEMORY_ERROR	存储器超限或不被支持的 PC 值，规定存储位置不存在或标签不支持 PC 值。
MEMORY_LOCKED	存储器锁定，存储位置锁定永久锁定，且不可写入。
ACCESS_PWD_ERROR	访问密码错误。
KILL_PWD_ERROR	销毁密码错误。
INVALID_KILL_PWD	销毁密码不能为全 0。
UNSUPPORTED_CMD	电子标签不支持该命令。
INVALID_ACCESS_PWD	对该命令，访问密码不能为 0。
READ_PROTECTED	电子标签已经被设置了读保护，不能再次设置。
NOT_READ_PROTECTED	电子标签没有被设置读保护，不需要解锁。
GENERIC_ERROR	非特定错误，标签不支持特定错误代码。
WRITE_FAILED	有字节空间被锁定，写入失败。
LOCK_FAILED	不能锁定。
ALREADY_LOCKED	已经锁定，不能再次锁定。
PARAM_SAVE_FAILED	参数保存失败,但设置的值在读写模块断电前有效。
ADJUST_FAILED	无法调整。
ANTENNA_ERROR	天线检测错误
CMD_EXEC_FAILED	命令执行出错。
COMMUNICATION_FAILURE	有电子标签，但通信不畅，无法操作。
NO_TAG	无电子标签可操作。
TAG_ERROR	电子标签返回错误代码。
INVALID_CMD_LENGTH	命令长度错误。
ILLEGAL_CMD	不合法的命令。
PARAM_ERROR	参数错误。
INVALID_INVENTORY_MODE	盘点模式(InventorySceneMode)非自定义模式，不允许设置。
UNSUPPORTED_FEATURE	固件不支持该功能。
COMMUNICATION_ERROR	通讯错误。

SDK_UNINITIALIZED	SDK 未初始化
DEVICE_DISCONNECTED	RFID 设备未连接(外接设备或蓝牙设备)
SEND_CMD_FAILED	发送指令数据失败(外接设备或蓝牙设备)

错误码也可以通过如下方式获取错误信息：

```
    /**
     * 获取错误码对应错误信息
     * @param errorCode 错误码
     * @return errorMessage 返回具体的错误信息
     */
    String errorMessage = RfidErrorConstants.getDescription(int errorCode);
```

附录 2：频段说明

欧标频段（865~868MHz）

频率区域	频段名称	频点个数	频道间隔 kHz	频点公式 MHz	范围（MHZ）
FrequencyRegion.UKRAINE	乌克兰频段	7	100	868.0 + N*0.1(MHz) 其中 N∈[0, 6]	868.0-868.6
FrequencyRegion.CHINESE1	中国标准 1 区	20	250	840.125 + N * 0.25 (MHz) 其中 N∈[0, 19]	840.125-844.875
FrequencyRegion.EU	欧洲频段	4	600	865.7 + N * 0.6(MHz) 其中 N∈[0, 3]	865.7-867.5

美标频段（902~928MHz）

频率区域	频段名称	频点个数	频道间隔 kHz	频点公式 MHz	范围（MHZ）
FrequencyRegion.CHINESE2	中国标准 2 区	20	250	920.125 + N * 0.25 (MHz) 其中 N∈[0, 19]	920.125-924.875
FrequencyRegion.US	美国频段	50	500	902.75 + N * 0.5 (MHz) 其中 N∈[0,49]	902.75-927.25
FrequencyRegion.KOREAN	韩国频段	6	600	917.3 + N * 0.6 (MHz) 其中 N∈[0,5]	917.3-920.3
FrequencyRegion.PERU	秘鲁	12	900	916.2 + N*0.9(MHz) 其中 N∈[0, 11]	916.2-926.1
FrequencyRegion.US3	美国工业/科学频段扩	53	500	902 + N * 0.5 (MHz) 其中 N	902-928

	展			∈[0,52]	
FrequencyRegion.Taiwan	中国台湾	12	500	922.25 + N * 0.5 (MHz) 其中 N∈[0,11]	922.25-927.75

设置这些特定区域频段的示例:

如欧洲频段 FrequencyRegion.EU：（从上到下依次对应参数 0-3）

865.7 0

866.3 1

866.9 2

867.5 3

如果需要设置欧洲频段 866.3-866.9

```
mRfidManager.setFrequencyRegion(FrequencyRegion.EU, 1, 2);
```

如果需要设置欧洲频段 865.7-867.5

```
mRfidManager.setFrequencyRegion(FrequencyRegion.EU, 0, 3);
```

附录 3: RSSI 参数对照表

RSSI 参数	信号强度	RSSI 参数	信号强度
110(0x6E)	-19dBm	70(0x46)	-60dBm
109(0x6D)	-20dBm	69(0x45)	-61dBm
108(0x6C)	-21dBm	68(0x44)	-62dBm
107(0x6B)	-22dBm	67(0x43)	-63dBm
106(0x6A)	-23dBm	66(0x42)	-64dBm
105(0x69)	-24dBm	65(0x41)	-65dBm
104(0x68)	-25dBm	64(0x40)	-66dBm
103(0x67)	-26dBm	63(0x3F)	-67dBm
102(0x66)	-27dBm	62(0x3E)	-68dBm
101(0x65)	-28dBm	61(0x3D)	-69dBm
100(0x64)	-29dBm	60(0x3C)	-70dBm
99(0x63)	-30dBm	59(0x3B)	-71dBm
98(0x62)	-31dBm	58(0x3A)	-72dBm
97(0x61)	-32dBm	57(0x39)	-73dBm
96(0x60)	-33dBm	56(0x38)	-74dBm
95(0x5F)	-34dBm	55(0x37)	-75dBm
94(0x5E)	-35dBm	54(0x36)	-76dBm
93(0x5D)	-36dBm	53(0x35)	-77dBm
92(0x5C)	-37dBm	52(0x34)	-78dBm
91(0x5B)	-38dBm	51(0x33)	-79dBm
90(0x5A)	-39dBm	50(0x32)	-80dBm
89(0x59)	-41dBm	49(0x31)	-81dBm
88(0x58)	-42dBm	48(0x30)	-82dBm
87(0x57)	-43dBm	47(0x2F)	-83dBm
86(0x56)	-44dBm	46(0x2E)	-84dBm
85(0x55)	-45dBm	45(0x2D)	-85dBm
84(0x54)	-46dBm	44(0x2C)	-86dBm
83(0x53)	-47dBm	43(0x2B)	-87dBm
82(0x52)	-48dBm	42(0x2A)	-88dBm
81(0x51)	-49dBm	41(0x29)	-89dBm
80(0x50)	-50dBm	40(0x28)	-90dBm
79(0x4F)	-51dBm	39(0x27)	-91dBm
78(0x4E)	-52dBm	38(0x26)	-92dBm
77(0x4D)	-53dBm	37(0x25)	-93dBm

76(0x4C)	-54dBm	36(0x24)	-94dBm
75(0x4B)	-55dBm	35(0x23)	-95dBm
74(0x4A)	-56dBm	34(0x22)	-96dBm
73(0x49)	-57dBm	33(0x21)	-97dBm
72(0x48)	-58dBm	32(0x20)	-98dBm
71(0x47)	-59dBm	31(0x1F)	-99dBm

可以使用工具类转换成 dBm 值：

int dBm = RssiToDBmUtils.dBmConver(int RSSI);
或通过方法 2.5.15setRssiInDbm(boolean isDbm)设置默认输出格式。

附录 4: 区域对照表

参数(CountryCode)	地区
ALB	Albania
ARG	Argentina
ARM	Armenia
AUT	Austria
AZE	Azerbaijan
BLR	Belarus
BEL	Belgium
BIH	Bosnia and Herzegovina
BRA	Brazil
BUL	Bulgaria
CAN	Canada
CHI	Chile
CHN	China
COL	Colombia
CRC	Costa Rica
CRO	Croatia
CYP	Cyprus
CZE	Czech Republic
DEN	Denmark
DOM	Dominican Republic
EST	Estonia
FIN	Finland
FRA	France
GER	Germany
GRE	Greece
HUN	Hungary
ISL	Iceland
IRI	Iran, Islamic Rep.
ITA	Italy
KOR	Korea, Rep.
LIE	Liechtenstein
LTU	Lithuania
LUX	Luxembourg
MKD	Macedonia, FYR
MLT	Malta
MEX	Mexico
MDA	Moldova

NED	Netherlands
NZL	New Zealand
NGR	Nigeria
NOR	Norway
PAN	Panama
PER	Peru
POL	Poland
POR	Portugal
ROM	Romania
RUS	Russian Federation
KSA	Saudi Arabia
SVK	Slovak Republic
SLO	Slovenia
RSA	South Africa
ESP	Spain
SWE	Sweden
SUI	Switzerland
HKG	Hong Kong
THA	Thailand
TUN	Tunisia
TUR	Turkey
UAE	United Arab Emirates
GBR	United Kingdom
USA	United States
URU	Uruguay
VEN	Venezuela, RB
UKR	Ukraine
MAS	Malaysia
SIN	Singapore
AUS	Australia
IND	India
VIE	Vietnam
ISR	Israel
INA	Indonesia
JPN	Japan
PHI	Philippines

附录 5: APP 打包的混淆规则

#RFID-SDK 混淆规则

```
-keep class com.rfid.**{*;}

-keep class com.rfiddevice.**{*;}

-keep class com.android.hw.**{*;}

-keep class com.ubx.**{*;}

-keep class com.iflytek.**{*;}

-keep class com.lib.**{*;}

-keep class android.device.**{*;}

-keep class android.content.**{*;}

-keep class android.os.**{*;}
```

备注

额外 API 说明

1. 长距（带手柄）设备控制扫描头出光(针对非蓝牙设备)

RFIDSDKManager.getInstance().enableScanHead(boolean isOpen);

说明：此方法是针对长距带手柄 RFID 设备，控制手柄上的按键，是否触发扫码头扫码

参数	类型	说明
isOpen	boolean	true,手柄按键触发扫码；false，手柄按键不触发扫码

2. 蓝牙设备，使用 NFC 连接方式时，读到的信息解析 MAC 地址

String mac = NFCUtils.getBleMac(NdefMessage ndefMessage);

说明：传入 NFC 拿到的 NdefMessage 对象，方法返回的是蓝牙设备的 MAC 地址，拿到地址后，可以传入 2.1.2.1 initBTToMac(Context context, String mac, InitListener listener); 方法中的 mac 传参。

参数	类型	说明
ndefMessage	NdefMessage	NFC 拿到的 NdefMessage 对象

建议

- init()方法和 release()方法成对出现，退出应用时调用 release()断开 RFID 连接。
- 根据错误码对照表进行故障处理。

联系与支持

有关最新的更新、示例代码和技术支持，请联系我们的支持团队。

文档结束。